

SUMMER INTERNSHIP
B. TECH 3rd YEAR PASSING STUDENTS

ADMIN PANEL AND BACK-OFFICE PLATFORM FOR A B2B HERB SUPPLY BUSINESS

Summer Internship Report

Submitted to

Sharda University



In partial fulfilment of the requirements of the award of the

Degree of Bachelor of Technology

in

Computer Science and Engineering

by

FAIZAN HASSAN

Enrollment No.: 2023486998 | Roll No.: 2301010331

Under the mentorship of

Dr. Yojna Arora

Assistant Professor

Department of Computer Science and Engineering

School of Computing Science & Engineering

Sharda University

Greater Noida

July, 2026

DECLARATION OF THE STUDENT

I hereby declare that the work presented in this report, titled “Admin Panel and Back-Office Platform for a B2B Herb Supply Business,” is an outcome of my own effort, carried out under the guidance of Dr. Yojna Arora, during my internship at Konkuwan Herbs Pvt. Ltd. The report is submitted to Sharda University in partial fulfilment of the Bachelor of Technology examination, 2026–27.

I further declare that this report, or any part of it, has not been submitted previously to this or any other university or institution for the award of any degree or diploma.

Faizan Hassan

Enrollment No.: 2023486998 | Roll No.: 2301010331

CERTIFICATE

This is to certify that Faizan Hassan, Enrollment No. 2023486998, Roll No. 2301010331, a student of B.Tech (Computer Science Engineering) at Sharda University, has successfully completed an internship at Konkuwan Herbs Pvt. Ltd., during which he designed and developed an admin panel and back-office management platform for the company's B2B medicinal herb supply business.

This report is a record of work carried out by him during the period from 25 May 2026 to 18 July 2026, submitted in partial fulfilment of the requirements for the Bachelor of Technology examination, 2026–27, at Sharda University.

Faizan Hassan (2023486998)

Dr. Yojna Arora
Assistant Professor
Department of Computer Science & Engineering

LIST OF TABLES

1. Table 1.1 — Internship Particulars	12
2. Table 1.2 — Phase-Wise Timeline	13
3. Table 3.1 — Summary of Existing System Gaps	15
4. Table 5.1 — Functional Requirements	17
5. Table 5.2 — Non-Functional Requirements	18
6. Table 6.1 — Technology Stack	19
7. Table 6.2 — Two-Gate Access Control	22
8. Table 7.2 — Table Descriptions	25
9. Table 8.1 — Challan Types	30
10. Table 9.1 — High-Severity Findings	48
11. Table 10.1 — Test Cases	53
12. Table 11.1 — Delivered Outcome	54
13. Table 13.1 — Challenges and Resolutions	57
14. Table 14.1 — Future Enhancements, Re-Prioritised	58

LIST OF FIGURES ii

Figure 5.1: Use case diagram of the Konkuwan Herbs admin platform	17
Figure 6.1: Layered architecture - client, edge, server middleware, and data layer	21
Figure 6.2: Request sequence with authentication and authorisation	21
Figure 6.3: Access control decision flow	22
Figure 7.1: Entity-relationship diagram of the core and farm-operations schema	23
Figure 7.2: Entity-relationship diagram, commerce schema	24
Figure 7.3: Entity-relationship diagram, supply schema	24
Figure 8.1: Public Home Page	27
Figure 8.2: Public Products Page with Category Filter	28
Figure 8.3: Buyer enquiry form with the multi-select product picker open	28
Figure 8.4: Admin Login Page	29
Figure 8.5: Financial-year period resolution, clamping and baseline trimming	29
Figure 8.6: Admin Dashboard, annual FY view, Severity-ranked “needs attention” alerts panel	30
Figure 8.7: Revenue by month, annual period and Day-by-day view after drilling into a month	31
Figure 8.8: Table of all products with reorder, name, botanical name, price range, and active	32
Figure 8.9: Modal form for creating or editing a product, showing the price range fields, HSN code, category assignment, and image upload options.	32
Figure 8.10: Order Management: Orders List with Status Filters	33
Figure 8.11: Order status state diagram, enforced server-side by the allowed-transitions map	33
Figure 8.12: Sequence diagram for setting a negotiated final price on an order line item	34
Figure 8.13: Order detail with line items and the “Set price” control.	34
Figure 8.14: Generated GST invoice PDF	35
Figure 8.15: Generated quotation PDF	35
Figure 8.16: Customer list with lead-status badge and Total Purchase column	36
Figure 8.17: Purchase-analytics bar chart, sortable.	36
Figure 8.18: Customer profile with order history	37
Figure 8.19: Challan type decision flow required fields and validation branches per type	37
Figure 8.20: Farmer → warehouse challan with an unregistered (“Other”) supplier.	38
Figure 8.21: Warehouse → warehouse stock transfer	38
Figure 8.22: Printed delivery challan	39
Figure 8.23: Warehouse registry	39
Figure 8.24: Cash position panel with the safe-to-deploy calculation	40
Figure 8.25: CropOS with the crop tab selector, week-in-cycle counter, and a generated Package of Practices	41
Figure 8.26: Generated War Room brief with overall status banner, crop status list, ranked actions, and risk radar	42
Figure 8.27: Farmer coverage bars per crop against enrolment targets, with crop filter buttons	43
Figure 8.28: Farmer profile and activity timeline	43
Figure 8.29: Stakeholder - Per-user dashboard widget grants	44

Figure 8.30: Restricted dashboard as seen by a stakeholder	44
Figure 8.31(a): Admin interface in Odia	45
Figure 8.31(b): Admin interface in Hindi	45
Figure 8.15: Audit Logs screen	46
Figure 8.16: Restricted dashboard as seen by a stakeholder	46

ACKNOWLEDGEMENT

This internship gave me my first real taste of building software that an actual business depends on, not a classroom exercise with a fixed rubric, but a working admin panel that the founders use to run their day-to-day operations. I am grateful for the chance to have built something that outlasted the internship itself.

I would like to thank Dr. Yojna Arora, my internal mentor at Sharda University, for her guidance throughout this internship and for pushing me to document the work properly rather than leaving it as scattered commits.

I am equally grateful to Mr. Rajeshwar Dhavala, Co-Founder and COO of Konkuwan Herbs Pvt. Ltd., who supervised my work on-site, trusted me with real production access from an early stage, and gave direct, practical feedback on every module I built. That trust shaped how I approached the project, since decisions had real consequences for people running an actual supply chain across seven states.

I also thank the broader team at Konkuwan Herbs for their patience while I learned the business before writing a line of code, and Sharda University for making this internship a part of the curriculum in the first place.

Finally, thanks to my family and friends for the usual reasons, putting up with late nights and half-finished sentences about Sequelize associations.

ABSTRACT

Konkuwan Herbs Pvt. Ltd. is a B2B medicinal-herb supply company that sources directly from farming families across several Indian states and sells to herbal manufacturers, exporters and processors. Before this project, the company operated through a static HTML website and informal record-keeping: buyer enquiries arrived by email, the product catalogue lived in markup, negotiated prices existed only in memory, procurement from farmers produced no document at all, and no record existed of who changed what.

This report documents the design, implementation, security audit and deployment of a full-stack platform built during a paid, on-site internship between 25 May and 18 July 2026, with subsequent hardening and feature work carried through to August 2026. The system replaces the static site with a React single-page application and adds sixteen administrative modules spanning three domains: **commerce** (catalogue, customers, orders with negotiated pricing, quotations and GST invoices), **procurement and logistics** (delivery challans in two distinct forms, warehouse registry), and **farm operations** (crop tracking with AI-generated field guidance, farmer enrolment and visit logging, farm finance, and an AI-generated weekly management brief).

Three design decisions distinguish the system from a conventional administrative panel. First, **negotiated pricing is modelled as a first-class database concept**: `order_items` carries both a quoted `unit_price` and an agreed `final_price`, with every change appended to a `pricing_history` table, making the commercial question “what did we quote versus what did we achieve?” answerable. Second, **all reporting follows the Indian financial year** (1 April – 31 March) with quarterly and monthly sub-periods, and every range is clamped so that no figure is ever reported beyond the current date, with the comparison baseline trimmed to the same elapsed span so that trend percentages remain meaningful. Third, an external **stakeholder role** can be granted individual dashboard metrics on a per-user basis, enforced by filtering the API response on the server rather than by hiding elements in the interface.

The platform is built with React 19 and Vite 8 on the client, Express 4 on Node.js for the API, and PostgreSQL through Supabase for data, authentication and file storage. It is deployed as a **single Cloudflare Worker** serving both the application and the API from one origin, with a Docker image provided as an alternative self-hosting target.

Notably, the system uses **no ORM**: all database access is performed over HTTPS through Supabase’s PostgreSQL interface, a constraint imposed by and a prerequisite for - the edge deployment target. Prior to deployment a structured pre-production audit was carried out across authentication, authorisation, the API surface, the database layer, file uploads, the frontend and the dependency tree. It identified two critical defects - a PostgreSQL filter-injection vector that allowed a scoped query to be widened to an unscoped one, and the return of raw database error text to unauthenticated clients - together with seven

high-severity issues. All were fixed and verified through fifteen structured checks. The interface is available in English, Odia and Hindi with full three-way string parity.

The work was carried out independently, with the author responsible for the complete design and implementation: database schema, authentication and authorisation model, REST API, client application, PDF document generation, internationalisation, security audit and deployment configuration.

TABLE OF CONTENTS

Declaration of the Student	ii
Certificate	iii
List of Tables	iv
List of Figures	v
Acknowledgement	vii
Abstract	viii
1. Introduction	12
1.1 About Konkuwan Herbs	12
1.2 Internship Details and Timeline	12
2. Problem Statement	14
3. Existing System Analysis	15
3.1 Limitation of the Static Website	15
3.2 Limitation of the Operational Workflow	15
4. Proposed System	16
5. Requirements Analysis	17
5.1 Functional Requirements	17
5.2 Non-Functional Requirements	18
6. System Architecture	19
6.1 Technology Stack	19
6.1.1 Why the ORM was Removed	20
6.2 Architecture Overview	20
6.3 Authentication and Access Control	22
6.3.1 The two gate model for stakeholders	22
7. Database Design	23
7.1 Entity-Relationship Diagram	23
7.2 Table Description	25
7.3 Design decision in the schema	26
8. Module Description and Implementation	27
8.1 Public Website	27
8.2 Authentication Module	28
8.3 Dashboard and Analytics	29
8.4 Revenue Chart and Drill-Down	30
8.5 Product Management	31
8.6 Order Management and Negotiated Pricing	32
8.7 Customer Relationship Management	35
8.8 Delivery Challans and Warehouses	37

8.9 Finance	40
8.10 Farm Operations - CropOS	41
8.11 Farm Operations – War Room	42
8.12 Farm Operations - FarmerOS	42
8.13 User Management and the Stakeholder Role	44
8.14 FinanceOS	45
8.15 Audit Logs and Settings	45
9. Security Audits	47
9.1 Method	47
9.2 Critical Findings	47
9.2.1 PostgREST filter injection	47
9.2.2 Database error disclosure	47
9.3 High-Severity Findings	48
9.4 A Finding that could not be fixed	48
9.5 Hardening applied	49
10. Testing	50
10.1 Testing Approach	50
10.2 Representative Test Cases	51
10.3 Defects found by testing	52
10.4 Limitations	52
10.5 Bug Fixing and Refinement	52
11. Results and Screenshots	53
11.1 Delivered Outcome	53
11.2 Summary of the Delivered System	53
12. Learning Outcomes	55
13. Challenges Faced	57
14. Future Enhancements	58
15. Conclusion	59
References	60

1. Introduction

1.1 About Konkuwan Herbs Pvt. Ltd.

Konkuwan Herbs Pvt. Ltd. is a B2B supplier of medicinal herbs, spices, and superfoods. The company sources raw material directly from over 2,500 farming families spread across seven Indian states, Jharkhand, Odisha, Chhattisgarh, Madhya Pradesh, West Bengal, Manipur, and Mizoram - and sells to herbal manufacturers, nutraceutical brands, and wellness companies in India, along with exporters in the EU. The company name comes from Konkuwa, the village in Jharkhand where the founders began working with farming communities in 2018.

The business is structured around removing intermediaries from the herb supply chain: Konkuwan manages cultivation planning, farmer support, harvest collection, drying and grading, and dispatch, rather than acting as a trader that simply buys and resells. This is the reason a Farm Operations module ended up being just as important to this internship as the customer-facing order management system - the company's day-to-day work happens as much in the field as it does in sales.

The company is registered as Konkuwan Herbs Pvt. Ltd. and is headquartered at Baseli Sahi, Puri, Odisha. It has received recognition and support from institutions including IRMA iSeed, IIMA Ventures, IIM Calcutta Innovation Park, WRI India's Land Accelerator, and the Harit Bharat Fund. The company's public website is available at konkuwanherbs.com.

1.2 Internship Details and Timeline

This was a paid, on-site internship, compensated at ₹10,000 per month, carried out independently — the admin panel and all of its modules were designed and built by a single intern with no co-developer on the project. I reported directly to Mr. Rajeshwar Dhavala, Co-Founder and Chief Operating Officer of Konkuwan Herbs, who supervised the work on a near-daily basis and reviewed each module as it was completed.

The work was carried out 5 to 6 days a week between 25 May 2026 and 18 July 2026. The internal mentor for academic supervision at Sharda University was Dr. Yojna Arora.

Internship Particulars

Particular	Detail
Organization	Konkuwan Herbs Pvt. Ltd.
Address	Baseli Sahi, Puri, Odisha
Website	konkuwanherbs.com
Role	Admin Panel Developer (Intern)
Mode	On-site

Particular	Detail
Duration	25 May 2026 – 18 July 2026 (8 weeks)
Working Days	5–6 days per week
Stipend	₹10,000 per month
External Supervisor	Mr. Rajeshwar Dhavala, Co-Founder & COO
Internal Mentor	Dr. Yojna Arora, Asst. Professor, Sharda University

Phase-Wise Timeline

Phase	Duration	Activities
1	25–27 May 2026	Onboarding: Understanding the company's sourcing, order, and farm-operation pipeline, requirement gathering, identification of key user roles and workflows
2	28–29 May 2026	Analysis of the existing static website: Identifying its limitations, proposing the admin panel and backend-driven architecture, defining major management modules
3	30 May – 5 June 2026	UI/UX planning: Wireframing, design review, planning dashboard, product, customer, order, quotation/invoice, and delivery-challan interfaces
4	6–19 June 2026	Core development: database/schema design, Supabase authentication, dashboard, product and category modules, customer profiles, custom products, initial API layer, and role-based access structure
5	20 June – 3 July 2026	Order management: Negotiated pricing, customer CRM, quotation/invoice workflows, delivery challan module, Farm Operations, data validation, warehouse-related workflows, and financial-year data considerations
6	4–11 July 2026	System testing: Bug fixing, UI refinement, performance optimization, validation of different challan types, set-password/authentication flow, stakeholder/user-role handling, and Hindi locale support
7	12–18 July 2026	Final refinements, security and access-control review: API and deployment optimization, documentation, Dockerization/deployment preparation, Cloudflare/edge-deployment considerations, and final pre-production readiness

2. Problem Statement

Konkuwan Herbs' operations, before this project, ran on a static website and a set of informal tools — spreadsheets, WhatsApp messages, and memory — that had not scaled with the business. The company could describe its sourcing model clearly in conversation, but almost nothing about how it actually ran day to day existed as a queryable system of record. Reviewing this during the early phases of the internship surfaced six specific gaps that directly shaped what the proposed system needed to do, described below.

1. **No product catalogue as data.** The website's product listings were hard-coded HTML. Adding a product or changing a price required editing source files — something the founders could not do themselves.
2. **No order system of record.** Order status existed only as a mental model held by whoever was handling that order at the time, with no shared view across the team.
3. **Negotiated prices were not recorded.** Konkuwan advertises a price range and negotiates a final price on nearly every order, but that final figure was never captured anywhere that could later be queried for revenue reporting.
4. **No customer relationship tracking.** Buyer companies, contacts, and lead status were tracked informally, if at all, making it difficult to separate live customers from prospects still being worked.
5. **No accountability trail.** If a price or order status changed, there was no record of who changed it or when.
6. **No procurement record.** When Konkuwan bought crop from a farmer, no document was produced. Nothing recorded what was purchased, at what rate, what the pickup and transport cost, or which warehouse received it. Because cost of goods was unknown, gross margin could not be calculated - the company could state its revenue but not its profit on any given consignment.
7. **No reporting on the business's actual calendar.** Indian companies report on a financial year running 1 April to 31 March. Any reporting the company attempted used calendar months, which meant figures could not be reconciled against filings or presented to investors without manual rework.

Taken together, these gaps meant Konkuwan could describe its business in conversation but could not query it, a problem that grows more expensive every month. A sourcing business of this scale continues to operate without a system of record.

3. Existing System Analysis

The system in place before this internship was a static website built with plain HTML and CSS. It was not connected to any database or backend service, and it had no admin-facing component at all. Reviewing it during Phase 1 and Phase 2 of the internship surfaced the following limitations, which directly shaped the scope of the proposed system.

3.1 Limitations of the Static Website

1. Content was hard-coded into HTML files. Adding a product, changing a price range, or swapping an image required editing source files directly — something the founders could not do themselves.
2. There was no concept of a product catalogue as data. Each product's name, description, and price range existed only as text inside the page markup.
3. No contact form submissions were stored anywhere, they were likely routed through a third-party form service with no record kept on the company's side.
4. The site had no login, no roles, and consequently no way to restrict who could change what.

3.2 Limitations of the Operational Workflow

1. Order status (draft, confirmed, dispatched, delivered) existed only as a mental model held by whoever was handling that order, with no shared view across the team.
2. Final negotiated prices, which differ from the listed price range on almost every order, were not recorded anywhere that could be queried later for revenue reporting.
3. Farm-side records — which farmers were enrolled, which crops were planted when, and how cash was being spent — were kept inconsistently, which made it difficult to get an accurate picture of the farm operation at any given point in time.
4. There was no audit trail. If a price or order status changed, there was no record of who changed it or when.

Table 3.1: Summary of Existing System Gaps

Area	Gap Identified
Product catalogue	Static HTML, no database, no admin edit capability
Orders	No system of record, tracked informally
Pricing	Negotiated final price not recorded anywhere
Customers	No CRM, contacts and lead status not tracked
Farm operations	No structured record of crops, farmers, or expenses
Access control	No login or role separation of any kind
Audit trail	No record of who changed what, or when
Procurement	No document for crop purchased from farmers, cost of goods unknown

Area	Gap Identified
Reporting calendar	Calendar months only, no financial-year reporting

4. Proposed System

The proposed system, agreed upon with Mr. Rajeshwar Dhavala during Phase 2 of the internship, was a single full-stack platform sitting behind the existing public website, built around a proper relational database rather than hard-coded pages. The public-facing site itself was rebuilt in React, keeping its existing visual design intact, so that it could later be served from the same codebase and styled consistently with the new admin interface.

The proposed system was organized around four broad areas, each of which became one or more modules in the final implementation:

1. **A database-backed product catalogue** replacing the static HTML pages, editable from the admin panel.
2. **An order and pricing module** recording the full lifecycle draft, confirmed, dispatched, delivered, cancelled and separately recording the final negotiated price per line item, with an append-only history.
3. **A customer relationship module** tracking buyer companies, contacts and lead status, with CSV import for bulk migration of existing contacts.
4. **A Farm Operations module** covering crop tracking, farmer enrolment and field visits, expense and revenue logging, and a weekly AI-generated management brief.
5. **A procurement and logistics module** producing a formal delivery challan for crop purchased from a farmer, and a second document type for stock moved between warehouses — so that cost of goods becomes a recorded figure.
6. **A financial-year reporting layer** so that every dashboard figure, chart and comparison follows 1 April – 31 March, with annual, quarterly and monthly views.

Three cross-cutting requirements were layered over all six: **role-based access control**, **a complete audit trail**, and **a multilingual interface**, since field staff in Puri are more comfortable working in Odia than in English.

5. Requirements Analysis

5.1 Functional Requirements

The functional requirements below were derived from the workflow gaps identified in Chapter 3 and confirmed with the company during the design review phase.

ID	Requirement
FR-1	Admin users must log in with role-based access across six roles: super_admin , product_manager , order_manager , farm_manager , viewer , stakeholder .
FR-2	Admins must be able to create , update , and archive products , including image upload and category assignment .
FR-3	Public visitors must be able to browse active products with filtering by category , without logging in.
FR-4	Admins must be able to create orders against a customer , add line items, and progress order status through a fixed sequence.
FR-5	Admins must be able to record a final negotiated price per order line item, separate from the listed price .
FR-6	The system must generate an invoice (and a downloadable PDF) reflecting the final negotiated total for a given order.
FR-7	Admins must be able to manage customer records , including a bulk CSV import with duplicate detection.
FR-8	Super admins must be able to invite , edit , and deactivate other admin users .
FR-9	The system must record an audit log entry for create , update , and delete actions on key entities.
FR-10	The Farm Operations module must support crop setup , weekly observation logging , AI-generated weekly task guidance , expense and revenue logging with receipt upload , farmer enrolment with field visit logging , and an AI-generated weekly status brief .
FR-11	The dashboard must display revenue, order and customer KPIs with trend indicators against the equivalent previous period .
FR-12	The system must generate a quotation document with its own numbering sequence, linkable to the resulting invoice.
FR-13	The system must record a delivery challan for crop procured from a farmer, capturing products, purchase rate, logistics charges and the receiving warehouse.
FR-14	The system must support a second challan type for stock transfer between warehouses , requiring both endpoints and rejecting a transfer to the same location.
FR-15	An order line item must be recordable without a catalogue product , by name, for one-off or seasonal crops.

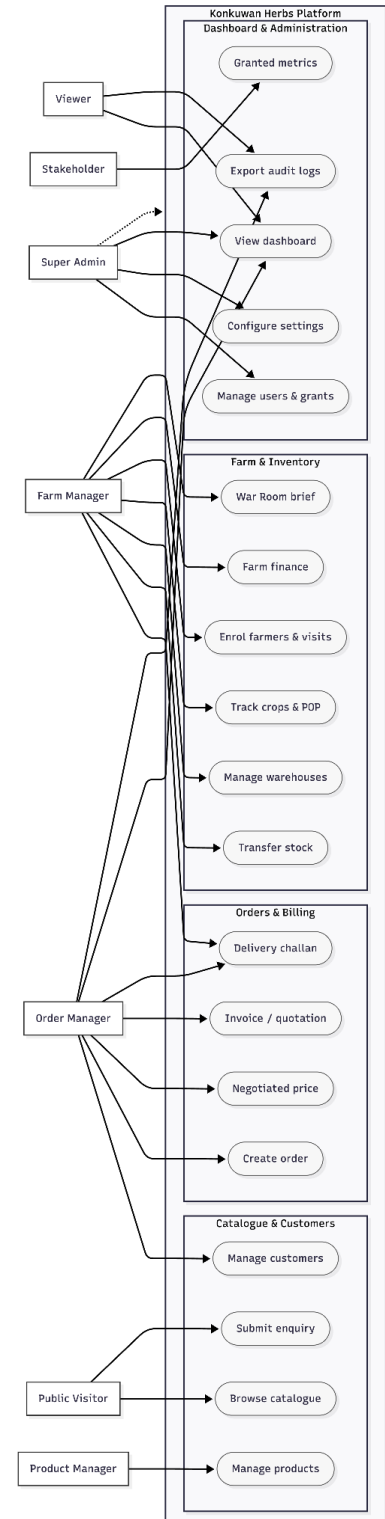


Figure 5.1: Use case diagram of the Konkuwan Herbs admin platform

ID	Requirement
FR-15	An order line item must be recordable without a catalogue product , by name, for one-off or seasonal crops.
FR-16	Dashboard reporting must operate on the Indian financial year , offering annual, quarterly and monthly views, and must never report data beyond the current date.
FR-17	A stakeholder role must exist that reaches only the dashboard, and within it only the metrics explicitly granted to that individual, enforced server-side.
FR-18	The administrative interface must be available in English, Odia and Hindi , with the preference stored per user.

5.2 Non-Functional Requirements

ID	Requirement
NFR-1	Security. Sessions handled by Supabase Auth. Every admin route must verify a valid session and the caller's role. Because the server uses a service-role key that bypasses row-level security, all database filters must be constructed from validated components rather than string interpolation.
NFR-2	Usability. The admin interface should require no developer involvement for routine tasks such as adding a product or updating an order status.
NFR-3	Performance. List views (products, orders, customers) must support pagination so that the interface does not slow down as data volume grows.
NFR-4	Maintainability. Clear separation between routes, controllers and validation. Any rule shared by two modules — billable order statuses, financial-year boundaries, role lists — must be defined in a single module and imported, never duplicated.
NFR-5	Compatibility. The admin panel must be usable on both desktop and mobile screens, since field staff may need to log expenses or farmer visits from a phone.
NFR-6	Data integrity. Database-level foreign key constraints must prevent orphaned records — for example, an order item referencing a deleted product.
NFR-7	Portability. The application must run unchanged on Cloudflare Workers and in a Docker container, which precludes any dependency on a persistent local filesystem or a long-lived database connection.
NFR-8	Error confidentiality. Server errors must not disclose database schema. Internal detail must be logged with a correlation identifier and withheld from the response.
NFR-9	Localisation. All user-facing strings must resolve through the i18n layer, with parity enforced across all supported locales

6. System Architecture

6.1 Technology Stack

The technology choices were made to keep the system within what a single developer could realistically build, test, and deploy within an eight-week internship, while still using tools suited to a small but real production workload.

Layer	Technology	Purpose
Frontend	React 19, Vite 8	Single-page application for both the public site and admin dashboard
Styling	Tailwind CSS v4	Utility-first styling, reused from the original site's design language
Routing	React Router v7	Client-side routing between public pages and protected admin routes
Data fetching	TanStack Query v5	Server-state caching, pagination, and mutation handling
Charts	Recharts 3	Dashboard Line, bar and pie charts with click-through interaction.
Internationalisation	i18next 26, react-i18next 17	Runtime language switching, persisted per user
PDF generation	jsPDF 4, jsPDF-AutoTable 5	Client-side invoice, quotation and challan documents
Backend runtime	Node.js, Express.js 4	REST API server
Validation	Joi 17	Request schema validation, including conditional rules per document type.
Security headers	Helmet 7	API response headers
ORM	Sequelize v6	Relational mapping for the core business schema
Data access	@supabase/supabase-js (PostgREST over HTTPS)	No ORM — see 6.1.1
Database	PostgreSQL (via Supabase)	Primary data store
Auth & Storage	Supabase Auth (JWT)	Authentication, session management, and file storage (receipts)
File uploads	Multer	Local disk handling for product images
Logging	Winston	Structured application logs
AI integration	Anthropic Claude API	POP generation and War Room brief generation
CSV handling	PapaParse	Bulk customer import

6.1.1 Why the ORM was removed

The original implementation used Sequelize v6. It was removed entirely during Phase 9. The reason is a hard constraint of the deployment target rather than a preference: **Cloudflare Workers cannot open a raw TCP connection to PostgreSQL** without either Cloudflare Hyperdrive or a container runtime. Sequelize requires a connection pool over TCP.

Supabase exposes the same PostgreSQL database over **HTTPS through PostgREST**, which a Worker can call natively. Migrating all data access to the Supabase client was therefore a prerequisite for edge deployment, not an optimisation.

The trade-off is explicit: the project loses ORM conveniences — model definitions, eager-loading helpers, migrations tied to models — and gains a deployment target with no cold-start database handshake and no connection-pool exhaustion. Relational integrity is preserved because it was never the ORM's responsibility: foreign keys and CHECK constraints are defined in PostgreSQL itself.

6.2 Architecture Overview

The system is a **single Cloudflare Worker** serving two kinds of traffic from one origin.

- Requests matching `/api/*` are routed through the Express application (`run_worker_first` in `wrangler.jsonc`).
- Everything else is served by **Workers Static Assets** — the compiled SPA — without executing any Node code, with `not_found_handling: "single-page-application"` so client-side routes resolve.

Serving both from one origin eliminates CORS configuration entirely and allows `VITE_API_URL` to default to a relative `/api` that is correct in development, preview and production alike. The cost is that frontend and backend release together, which is acceptable for a single-team internal system.

Express runs on Workers through `httpServerHandler` from `cloudflare:node`, which became viable when Workers gained `node:http` support. `worker.js` is the entry point:

```
import { httpServerHandler } from
'cloudflare:node';

import app from './server/src/app.js';

const PORT = 8080;

app.listen(PORT);

export default httpServerHandler({ port: PORT });
```

Figure 6.1: Layered architecture - client, edge, server middleware, and data layer of the Konkuwan Herbs admin platform

The diagram illustrates a layered architecture for the Konkuwan Herbs admin platform. It is organized into four main layers:

- Browser Layer:** Contains three components:
 - Public Site (Home, Products, Contact)
 - Admin SPA (React 19 + TanStack Query)
 - jsPDF (invoice, quotation, challan)
- Cloudflare Worker Layer:** Labeled "one origin", it contains:
 - Static Assets (SPA bundle + _headers)
 - Express 4 API (run_worker_first: /api/*)
- Server middleware Layer:** Contains:
 - authenticate (verify Supabase JWT)
 - errorHandler (redacts 5xx)
 - authorize(...roles)
 - Joi validation
- Supabase Layer:** Contains:
 - PostgreSQL
 - Storage
 - Auth

Interactions and data flow are indicated by arrows:

- The **Browser** sends requests to the **Cloudflare Worker** and **Supabase**.
- The **Cloudflare Worker** serves **Static Assets** and routes API requests to the **Express 4 API**.
- The **Express 4 API** passes requests through the **Server middleware**.
- The **Server middleware** interacts with **Supabase** for authentication, authorization, and data storage/retrieval.
- The **Supabase** layer includes **PostgreSQL** for data, **Storage** for files, and **Auth** for user management.
- External services like **OpenAI / Anthropic** are integrated into the **Express 4 API**.

Figure 6.2: Request sequence with authentication and authorisation

The diagram shows a sequence of messages between the Browser, Worker (Static Assets), Express, auth middleware, and Supabase:

- Browser** sends `GET /admin/orders` to **Worker (Static Assets)**.
- Worker (Static Assets)** returns `index.html + bundle (CSP from _headers)` to **Browser**.
- Browser** sends `GET /api/admin/orders (Bearer token)` to **Express**.
- Express** sends `authenticate` to **auth middleware**.
- auth middleware** sends `supabase.auth.getSession()` to **Supabase**.
- Supabase** returns `access_token` to **auth middleware**.
- auth middleware** sends `auth.getUser(token)` to **Supabase**.
- Supabase** returns `user` to **auth middleware**.
- auth middleware** sends `select profiles where id = user.id` to **Supabase**.
- Supabase** returns `profile (role, is_active)` to **auth middleware**.
- auth middleware** sends `[inactive or no profile]` to **Express**.
- Express** returns `401` to **Browser**.
- auth middleware** sends `authorise('super_admin', 'order_manager')` to **Express**.
- Express** returns `[role not permitted]` to **auth middleware**.
- auth middleware** sends `PostgreSQL query (service role)` to **Supabase**.
- Supabase** returns `rows` to **auth middleware**.
- auth middleware** returns `200 JSON` to **Browser**.

Figure 6.2: Request sequence with authentication and authorisation

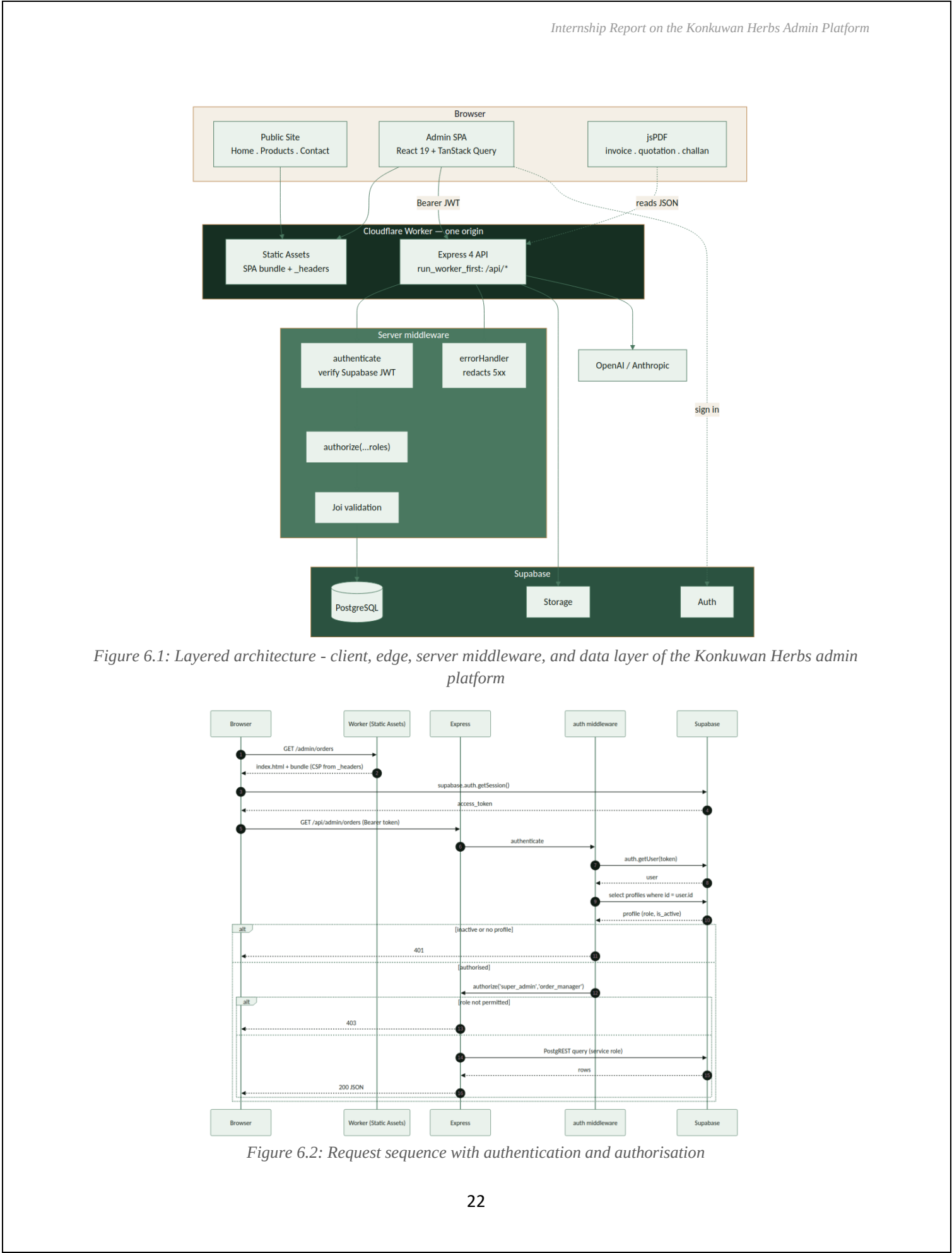


Figure 6.1: Layered architecture - client, edge, server middleware, and data layer of the Konkuwan Herbs admin platform

The diagram illustrates a layered architecture for the Konkuwan Herbs admin platform. It is organized into four main layers:

- Browser Layer:** Contains the Public Site (Home, Products, Contact), Admin SPA (React 19 + TanStack Query), and jsPDF (invoice, quotation, challan).
- Cloudflare Worker Layer:** Acts as a single origin, containing Static Assets (SPA bundle + _headers) and Express 4 API (run_worker_first: /api/*).
- Server middleware Layer:** Contains authentication (verify Supabase JWT), error handling (redacts 5xx), authorization (authorize(...roles)), and Joi validation.
- Supabase Layer:** Contains PostgreSQL, Storage, and Auth.

Arrows indicate the flow of data and requests between these layers. The Browser layer interacts with the Cloudflare Worker layer. The Cloudflare Worker layer interacts with the Server middleware layer. The Server middleware layer interacts with the Supabase layer. The Supabase layer interacts with the Cloudflare Worker layer. The Browser layer also interacts directly with the Supabase layer for authentication and data retrieval.

Figure 6.2: Request sequence with authentication and authorisation

The diagram shows a sequence of requests and responses between the Browser, Worker (Static Assets), Express, auth middleware, and Supabase. The sequence is as follows:

- The Browser sends a GET request to /admin/orders to the Worker (Static Assets).
- The Worker (Static Assets) returns index.html + bundle (CSP from _headers) to the Browser.
- The Browser sends a GET request to /api/admin/orders (Bearer token) to the Express layer.
- The Express layer sends a supabase.auth.getSession() request to the auth middleware.
- The auth middleware returns access_token to the Express layer.
- The Express layer sends an authenticate request to the auth middleware.
- The auth middleware sends an auth.getUser(token) request to Supabase.
- Supabase returns user to the auth middleware.
- The auth middleware sends a select profiles where id = user.id request to Supabase.
- Supabase returns profile (role, is_active) to the auth middleware.
- The auth middleware returns [inactive or no profile] to the Express layer.
- The Express layer returns 401 to the Browser.
- The Browser sends an alt request to the Express layer.
- The Express layer sends an authorize('super_admin', 'order_manager') request to the auth middleware.
- The auth middleware returns (role not permitted) to the Express layer.
- The Express layer returns 403 to the Browser.
- The Browser sends an alt request to the Express layer.
- The Express layer sends a PostgREST query (service role) to Supabase.
- Supabase returns rows to the Express layer.
- The Express layer returns 200 JSON to the Browser.

On the backend, each feature follows the same pattern: a **route file** declares the URL paths and attaches the relevant middleware, a **controller** validates input through a Joi schema and contains the business logic, **shared utilities** hold any rule used by more than one module. There are no model files, queries are written against the Supabase client directly in controllers. Public routes (product listing, category listing) skip the authentication middleware entirely, admin routes always pass through it first.

6.3 Authentication and Access Control

Authentication is delegated to Supabase rather than built in-house. On login, `supabase.auth.signInWithPassword` returns a session containing an access token, an Axios request interceptor attaches it as a Bearer header to every subsequent API call.

Server-side, the `authenticate` middleware:

1. verifies the token with `supabase.auth.getUser(token)`
2. loads the corresponding `profiles` row
3. rejects the request if the profile is missing or `is_active` is false
4. attaches `req.user = { ...user, profile }`.

Authorisation is a second middleware, `authorize(...roles)`, given the roles permitted on that route. A request from a role not in the list receives 403.

6.3.1 The two-gate model for stakeholders

The **stakeholder** role required a mechanism the route-level check cannot express, two stakeholders may hold the same role but be entitled to *different metrics*. This is implemented as two independent gates:

Gate	Question it answers	Implementation
Route gate	Which endpoints exist for this role?	<code>authorize(...roles)</code> — stakeholders reach <code>/api/admin/analytics/dashboard</code> and nothing else
Field gate	Which fields come back from that endpoint?	<code>filterDashboardForWidgets()</code> reduces the payload to the widgets granted to that individual

Neither is sufficient alone. A route check cannot express per-user field grants, a field filter applied on an unauthorised route is not a gate at all. Filtering happens on the server: hiding a card in the interface while the API still returns the number is not a permission, because the response is visible in the browser's network inspector.

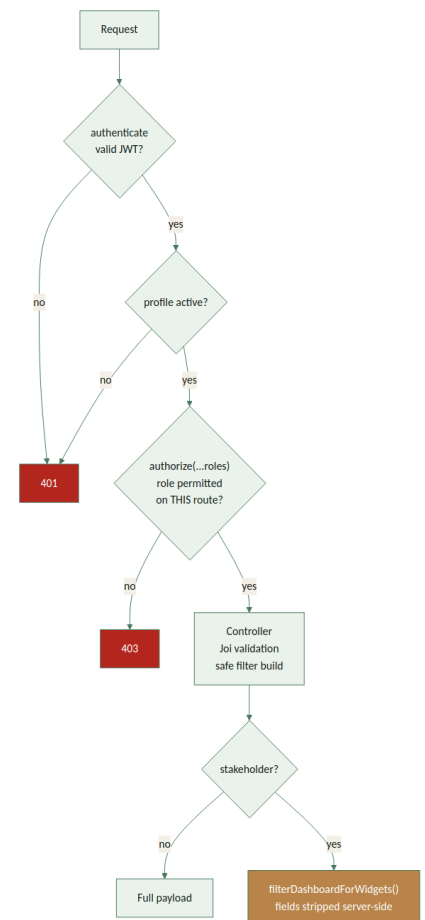


Figure 6.3: Access control decision flow route gate then field gate

7. Database Design

7.1 Entity-Relationship Diagram

The database is built on PostgreSQL, managed through Supabase, and accessed from the backend through Sequelize models. The schema is split into two groups of tables: the core business schema (products, orders, customers, pricing, and audit logs) and the Farm Operations schema (crop setups, observations, farmers, expenses, and cash balance). Both groups share a single profiles table, which extends Supabase's built-in auth.users table with a role and an active flag, rather than maintaining a separate users table.

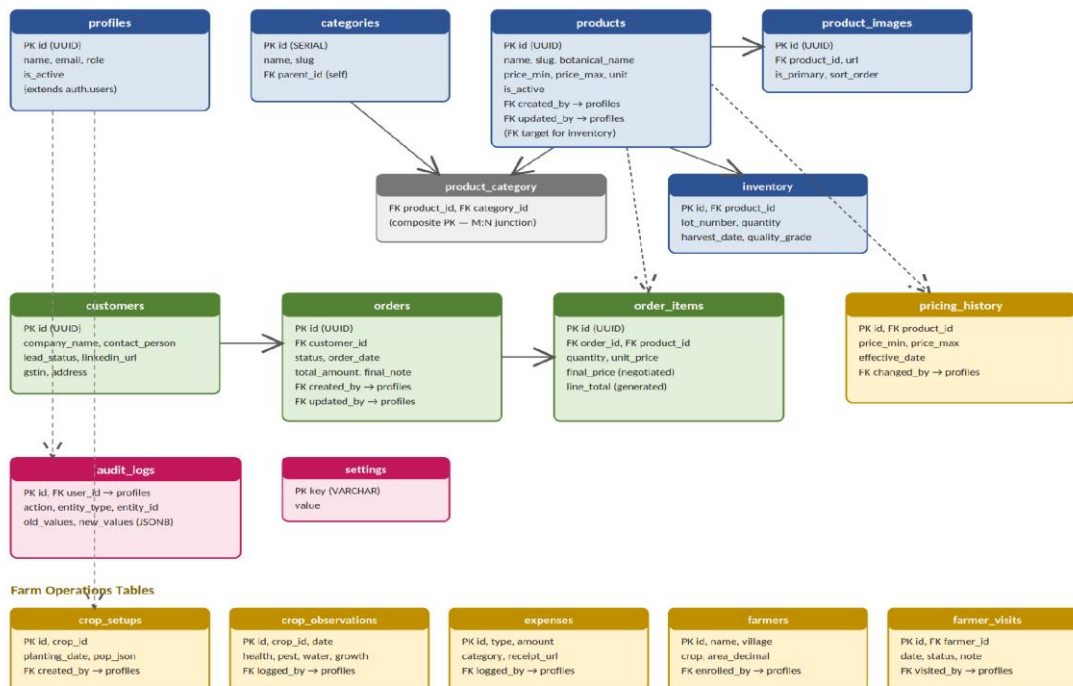


Figure 7.1: Entity-relationship diagram of the core and farm-operations schema

Twenty-three tables are actively queried, in three groups:

Group	Tables
Identity & governance	profiles, audit_logs, settings
Commerce	categories, products, product_images, product_category, inventory, customers, orders, order_items, pricing_history, contact_submissions
Supply & farm operations	warehouses, delivery_challans, challan_items, farmers, farmer_visits, crop_setups, crop_observations, expenses, cash_balance, war_room_briefs

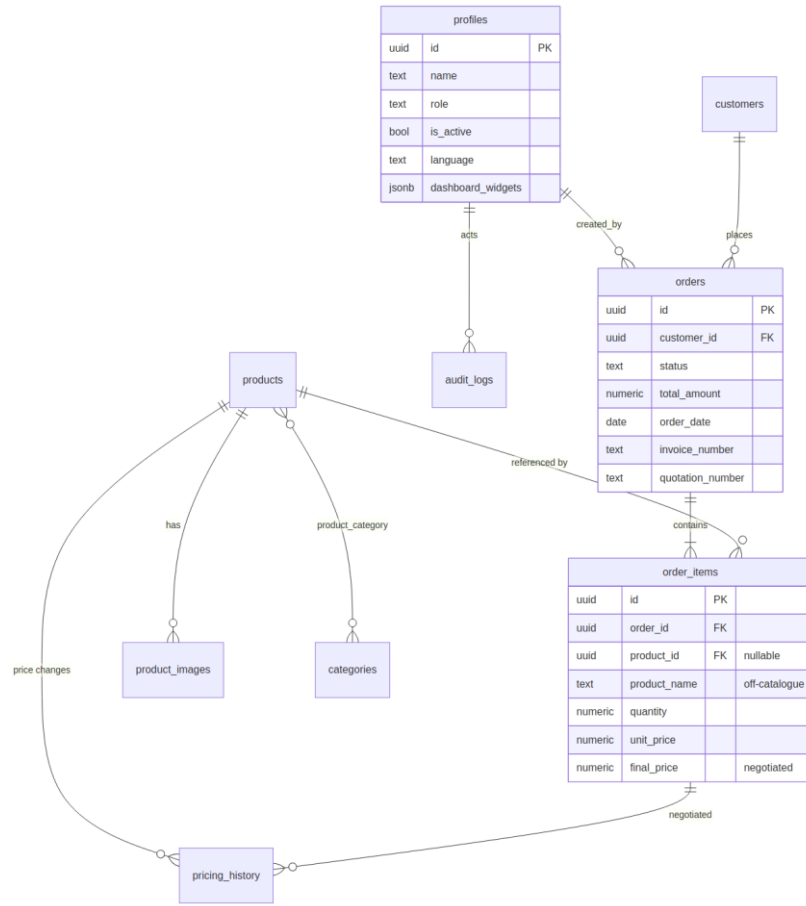


Figure 7.2: Entity-relationship diagram, commerce schema

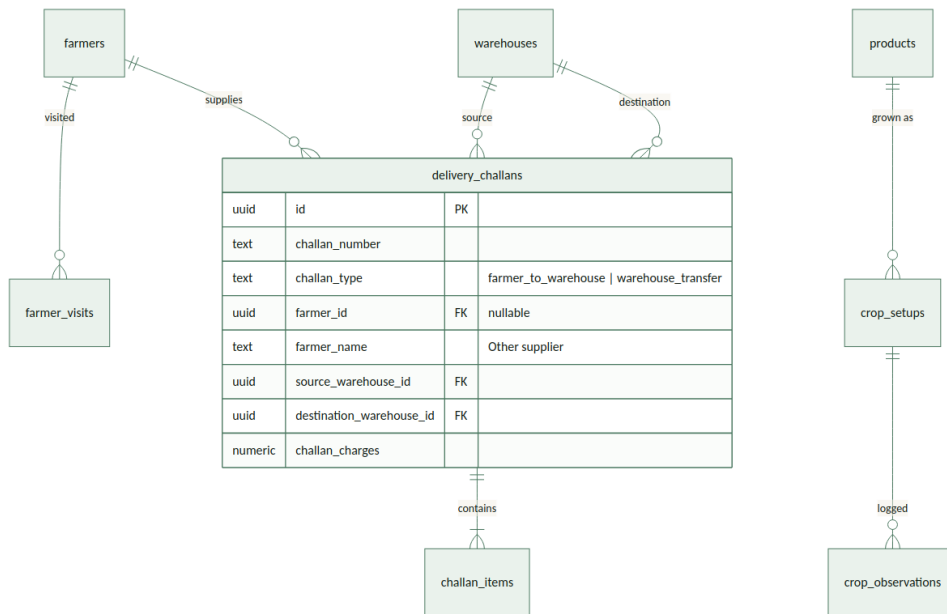


Figure 7.3: Entity-relationship diagram, supply schema

7.2 Table Descriptions

Table	Description
profiles	Extends Supabase auth.users with name, role, and is_active. The single source of identity for every other table's created_by / updated_by / logged_by foreign keys.
categories	Product categories, self-referencing through parent_id to allow a simple two-level hierarchy.
products	Core product catalogue: name, botanical name, description, available forms, price_min / price_max range, and unit (kg, MT, etc.).
product_images	One-to-many images per product, with an is_primary flag and a sort_order for gallery ordering.
product_category	Many-to-many junction table linking products to one or more categories.
inventory	Optional lot-level stock tracking per product (quantity, harvest date, quality grade).
customers	Buyer companies with contact details, GSTIN, lead_status (active_customer / potential_lead), and an optional LinkedIn URL.
contact_submissions	Public enquiry captures from the buyer and investor forms, with a status workflow (new → read → replied → archived).
warehouses	Storage locations: name, code, address, city, state, PIN, contact person, is_active. Inactive warehouses are hidden from new challans but retain their history
delivery_challans	(new) Procurement and transfer documents. challan_type is either farmer_to_warehouse or warehouse_transfer. A CHECK constraint (delivery_challans_type_shape) enforces the correct shape for each: a farmer challan requires either farmer_id or a free-text farmer_name; a transfer requires both warehouse ids and requires that they differ.
challan_items	Line items per challan: product (by id or free-text name), quantity, unit, purchase rate
orders	One order per customer transaction, with a status field constrained to draft, confirmed, dispatched, delivered, or cancelled.
order_items	Line items per order. Stores both the quoted unit_price and an optional final_price representing the negotiated amount, line_total is a database-generated column.
pricing_history	An append-only log of every price range change and every negotiated final price, tied to the product and the admin who made the change.
audit_logs	Generic audit trail recording the action, entity type, entity id, and JSONB snapshots of the old and new values for any tracked mutation.
settings	A simple key-value store for system-wide configuration values.
crop_setups	One row per crop being grown, storing the planting date and the latest AI-generated POP as JSONB.
crop_observations	Weekly field observations per crop — health, pest pressure, water status, and growth stage.
expenses	A single table for both expenses and revenue (distinguished by a type column), with an optional receipt_url pointing to Supabase Storage.
farmers	Enrolled farmers, their village, assigned crop, and cultivated area in decimals.

Table	Description
farmer_visits	Field visit log per farmer, recording status and free-text notes.
cash_balance	A running record of the farm operation's cash position, updated as a new row each time rather than overwritten.
war_room_briefs	Stores each AI-generated weekly status brief as JSONB, tied to the week it was generated for.

7.3 Design decisions in the schema

Negotiated pricing. `order_items` carries `unit_price` (quoted) and `final_price` (agreed) as separate columns, and every change appends a row to `pricing_history`. The alternative of overwriting the price would make it impossible to answer “what did we quote versus what did we achieve?”, which is the central commercial question of a negotiation-driven business.

Two workflows, one table. Procurement and stock transfer share `delivery_challans` because both produce a document listing goods moving between two points. Their required fields differ, which is enforced at two levels: a conditional Joi schema at the API boundary, and a CHECK constraint in the database. The database constraint is the one that matters — it holds even if a future code path forgets the validation.

Expenses and revenue in one table. `expenses` holds both, distinguished by a `type` column, because the farm-side ledger needs them in a single chronological sequence.

Cash position is append-only. `cash_balance` gains a row on each update rather than overwriting, so the history of who changed the figure and when is preserved.

8. Module Description and Implementation

This chapter walks through each module of the platform, describing what it does, how it was implemented, and which parts of the codebase it corresponds to. Modules are presented roughly in the order they were built, which also matches the phase-wise timeline in Chapter 1.

8.1 Public Website

The public website existed before this internship as a static HTML and CSS site. One of the first tasks, carried out during Phase 3 and the early part of Phase 4, was to rebuild it as a React application without changing how it looked. The visual design, the forest-green and cream colour palette, the typography, the section layout on the Home, Products, Supply, Impact, Partners, About, and Contact pages, was kept identical to the original. What changed underneath was the implementation.

The rebuilt site fetches its product catalogue from the new `/api/products` and `/api/categories` endpoints through the `useProducts` and `useCategories` hooks, both built on TanStack Query for caching. This means that products shown on the public Home and Products pages now come directly from the same database the admin panel writes to, rather than being hard-coded into the page markup as before. The Contact page still presents two separate forms (buyer inquiry and investor inquiry), matching the original design, the buyer enquiry form now offers a **searchable multi-select product picker** rather than a single free-text field, so an enquiry arrives with structured product interest.



Figure 8.1.1: Public Home Page

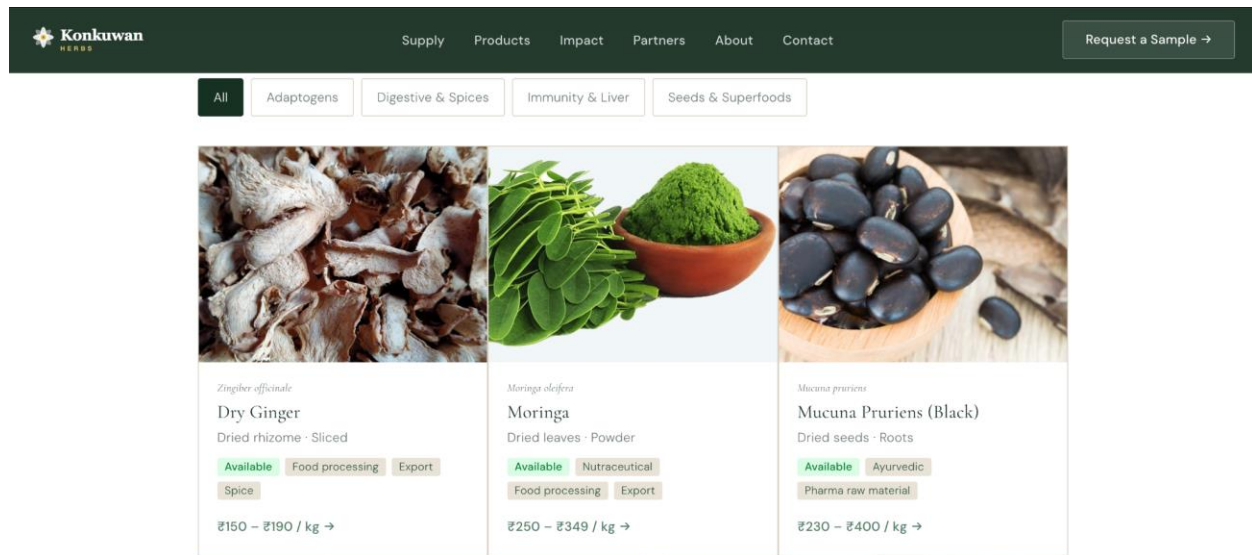


Figure 8.1.2: Public Products Page with Category Filter

FOR BUYERS

Source medicinal herbs, spices, or superfoods.

Tell us what you need and we'll come back with availability, grades, and a sample plan.

Request a Sample

48h
Response time

FSSAI
Compliant supply

Farm-direct
Traceable lots

FOR INVESTORS & PARTNERS

Investing or partnering?

Get our investor brief with financials, farm network, and expansion plans.

Request Investor Brief

✉ info@konkuwanherbs.com

☎ +91 8809 227099

📍 Baseli Sahi, Puri, Odisha 752001

Figure 8.1.3: Buyer enquiry form with the multi-select product picker open

8.2 Authentication Module

The admin side of the application sits behind a login screen at `/admin/login`. Authentication is implemented through Supabase rather than a custom solution: the Login component calls `supabase.auth.signInWithPassword()`, and on success the resulting session and access token are stored in an AuthContext that wraps the whole admin section of the application. The ProtectedRoute component checks this context before rendering any admin page, redirecting to the login screen if no

valid session is present. Every subsequent API call from the frontend carries the access token as a Bearer header, attached automatically by an Axios request interceptor in **services/api.js**. If the backend responds with a 401, a response interceptor signs the user out and redirects to the login page, so an expired session does not leave the interface in a broken state. Invited users now land on a dedicated **set-password page**, account holders can change their password from **My Account**, which also carries the language preference.

8.3 Dashboard and Analytics

The dashboard is driven by a single endpoint, GET **/api/admin/analytics/dashboard**, which accepts period (year | quarter | month), fy, and either quarter or month.

The financial-year resolver. All period arithmetic lives in **utils/financialYear.js**, which resolves a request into concrete bounds plus the equivalent previous period. Quarters follow the Indian convention: Q1 Apr–Jun, Q2 Jul–Sep, Q3 Oct–Dec, Q4 Jan–Mar of the following calendar year.

Clamping to the current date. Every range is trimmed so that no figure is reported beyond today. Two problems made this necessary:

1. **Reporting.** On 6 August, an annual view queried through 31 March of the following year, so the revenue chart drew eight empty future months and the line fell to zero for the rest of the year.
2. **Comparison.** More seriously, five months of the current year were being compared against a **full twelve months** of the previous year, which made every annual trend percentage meaningless.

The fix clamps the reporting range to today **and trims the comparison baseline to the same elapsed number of days**, so 1 Apr – 6 Aug 2026 is compared against 1 Apr – 6 Aug 2025. The interface displays “as of 6 Aug 2026” whenever a period is still in progress. A period entirely in the future yields an empty range rather than a page of zeroes.

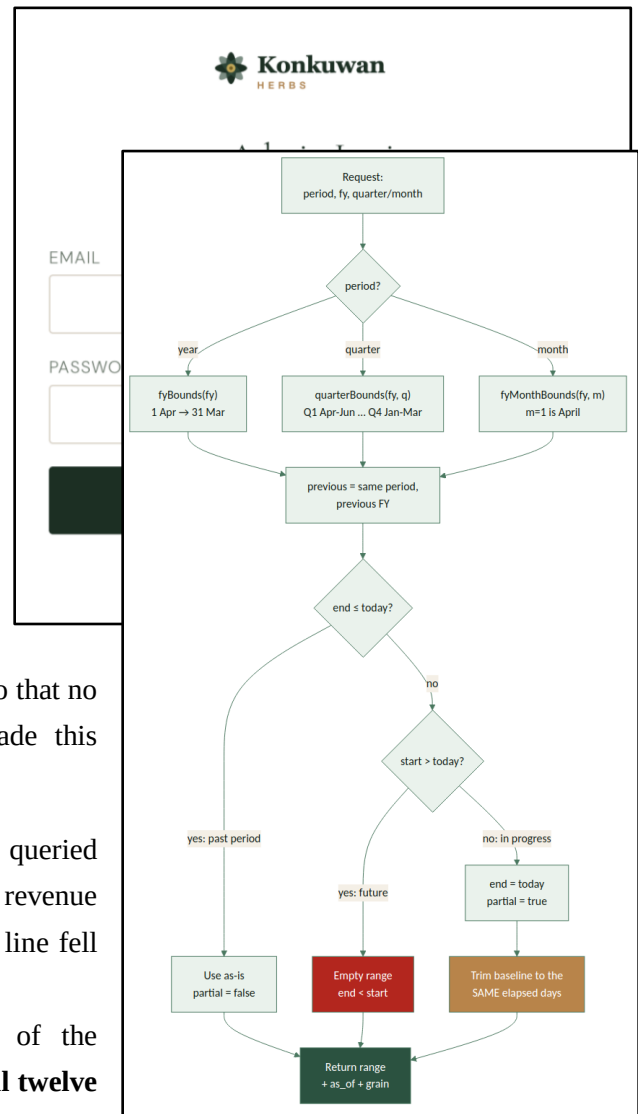


Figure 8.3.1: Financial-year period resolution, clamping and baseline trimming

Widget synchronisation. Every widget follows the period selector. One did not: the order-pipeline chart queried every order ever written, making it the only element on the page that never changed when the period was switched. It is now derived from the period-scoped result. The “draft orders” figure in the alerts panel is deliberately not period-scoped, it asks “what is unconfirmed right now,” and a draft from March is still unconfirmed in August.

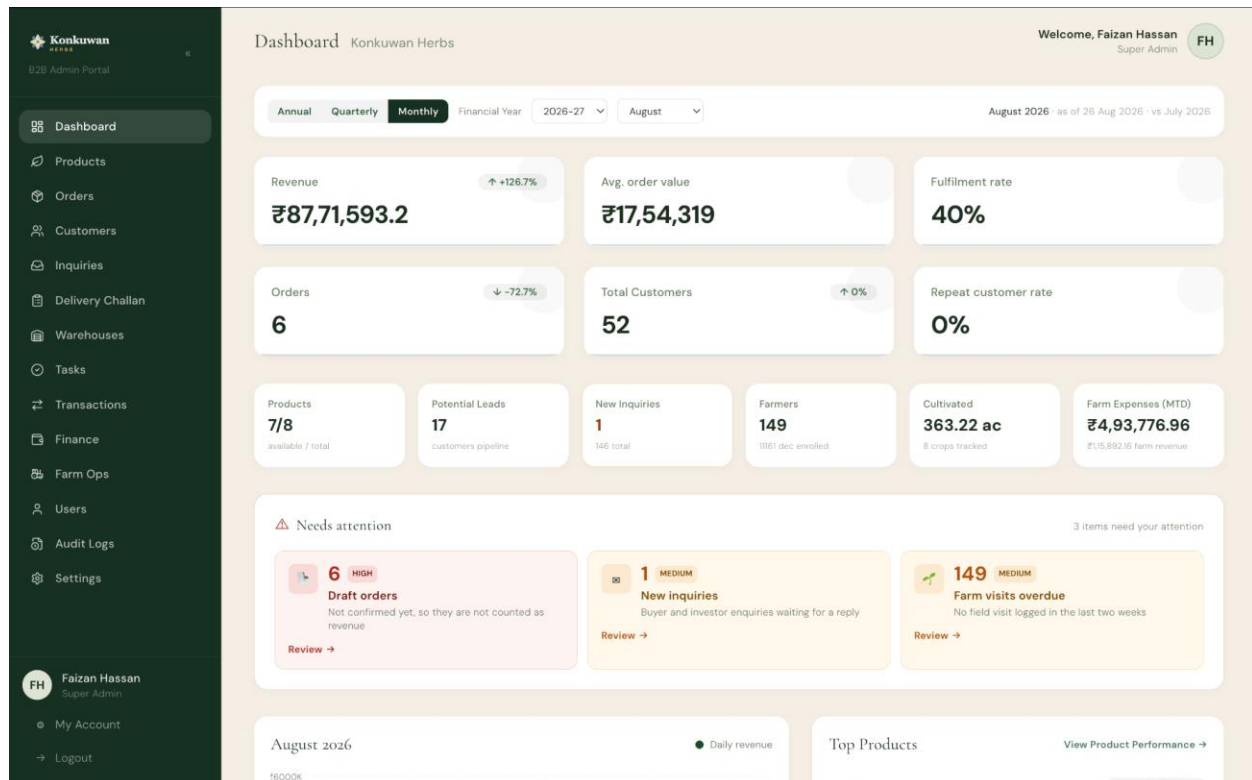


Figure 8.3.2: Admin Dashboard, annual FY view, Severity-ranked “needs attention” alerts panel

8.4 Revenue Chart and Drill-Down

The revenue chart supports two granularities and a drill-down between them.

Why it had not worked. The chart always bucketed revenue by month. In a monthly reporting period that produces exactly one data point — so the chart rendered a single dot, and clicking it “drilled into” the month already displayed. The interaction appeared broken because, functionally, it was.

The fix follows the grain field the server returns:

Period	Technology	Clickable?
Annual	up to 12 months	Yes → that month, day by day
Quarterly	up to 3 months	Yes → that month, day by day
Monthly	the days	No — the days are the view

Both series derive from **one query**, so drilling makes no additional request. The logic is extracted into `client/src/lib/revenueSeries.js` as pure functions, which makes it testable independently of React.

Further refinements: month labels carry the year when a financial year spans two calendar years (so Jan is unambiguous), the tooltip shows exact rupee figures while the axis stays abbreviated, day ticks use `preserveStartEnd` so thirty-one of them fit on a phone, and a drill-down opened in one financial year is cleared when the period changes

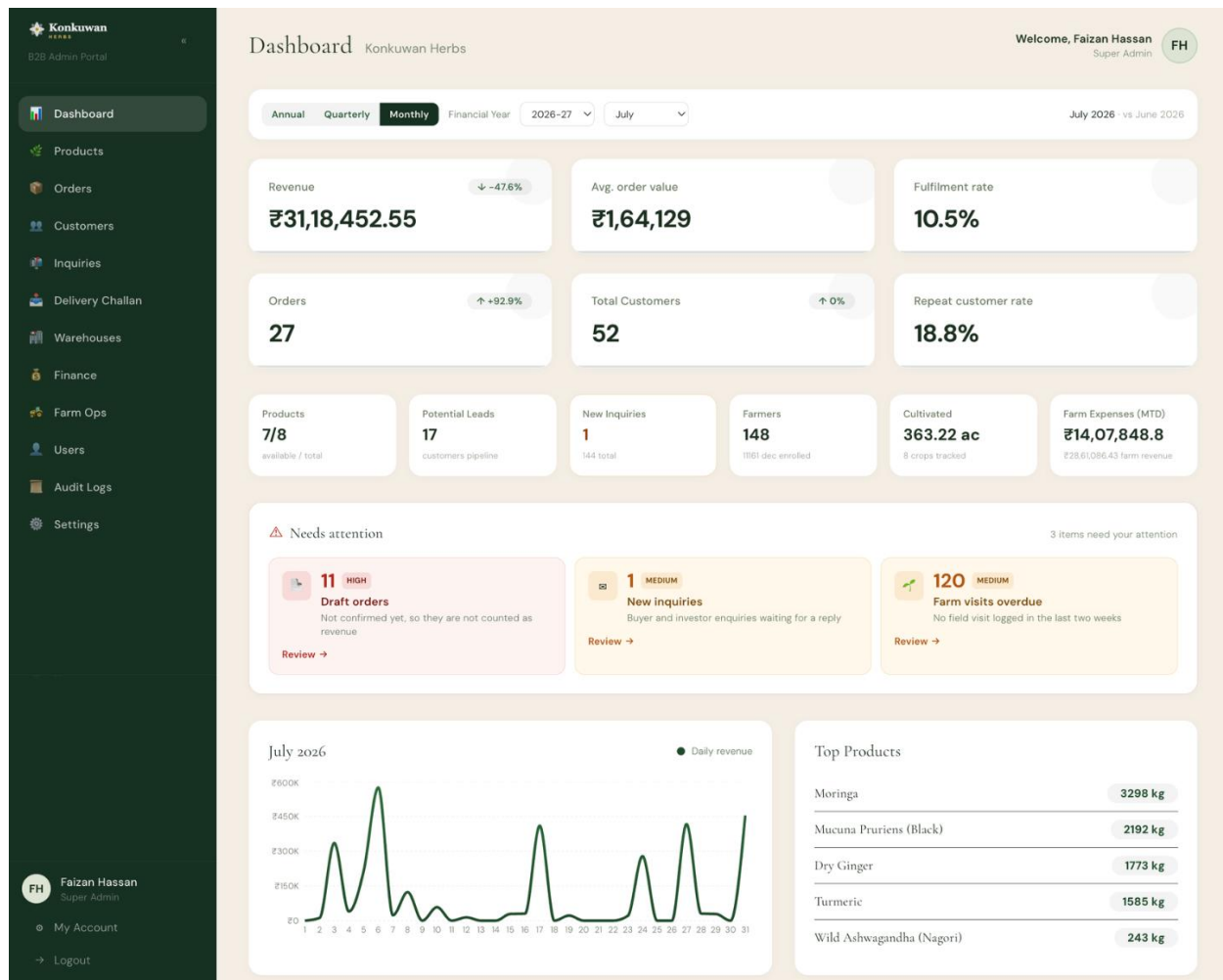


Figure 8.3.2: Revenue by month, annual period and Day-by-day view after drilling into a month

8.5 Product Management

The product management screen lists every product through a paginated `DataTable` component backed by `GET /api/products` (the same endpoint used by the public site, since the admin list simply requests a larger page size). Creating or editing a product opens a modal form covering name, botanical name, description, available forms, unit, and the `price_min` / `price_max` range, leaving both price fields

blank causes the public site to display “Available on inquiry” instead of a price range, which matches how Konkuwan actually prices several of its less common items.

NAME	BOTANICAL	PRICE RANGE	ACTIVE	ACTIONS
Dry Ginger	Zingiber officinale	₹150 – ₹190 / kg	Active	Archive Delete
Moringa	Moringa oleifera	₹250 – ₹349 / kg	Active	Archive Delete
Mucuna Pruriens (Black)	Mucuna pruriens	₹230 – ₹400 / kg	Active	Archive Delete
Safed Musli	Chlorophytum borivilianum	On inquiry	Active	Archive Delete
Turmeric	Curcuma longa	On inquiry	Active	Archive Delete
Giloy	Tinospora cordifolia	₹80 – ₹120 / kg	Archived	Unarchive Delete
Kalmegh	Andrographis paniculata	On inquiry	Active	Archive Delete
Holy Basil (Tulsi)	Ocimum tenuiflorum	On inquiry	Active	Archive Delete

Figure 8.4.1: Table of all products with reorder, name, botanical name, price range, and active/archived status

Products carry an **HSN/SAC code**, printed on invoices for GST compliance. Image handling supports three paths at once: direct file upload, selection from a small bundled asset gallery, or pasting an external image URL, all converging on the same `product_images` table with an `is_primary` flag that can be toggled per image. Image upload now validates the **magic bytes** of the file rather than the client-declared MIME type, and stores images in **Supabase Storage** rather than on local disk. On the backend, creating or updating a product also generates a unique slug from the product name (falling back to a timestamp suffix if the slug already exists) and, when the price range changes, writes a row into `pricing_history` so that price changes over time remain queryable.

8.6 Order Management and Negotiated Pricing

Orders are the most workflow-heavy part of the system, since Konkuwan's business model depends on negotiating a final price for each order rather than selling strictly at the listed price range. An order is created against an existing customer with one or more line items, each referencing a product, a quantity, and a starting `unit_price`. The order's status then moves through a fixed sequence, shown in Figure 8.1, which is enforced on the backend through an explicit map of allowed transitions in `order.controller.js`, so an order cannot, for example, jump directly from draft to delivered.

Figure 8.4.2: Modal form for creating or editing a product, showing the price range fields, HSN code, category assignment, and image

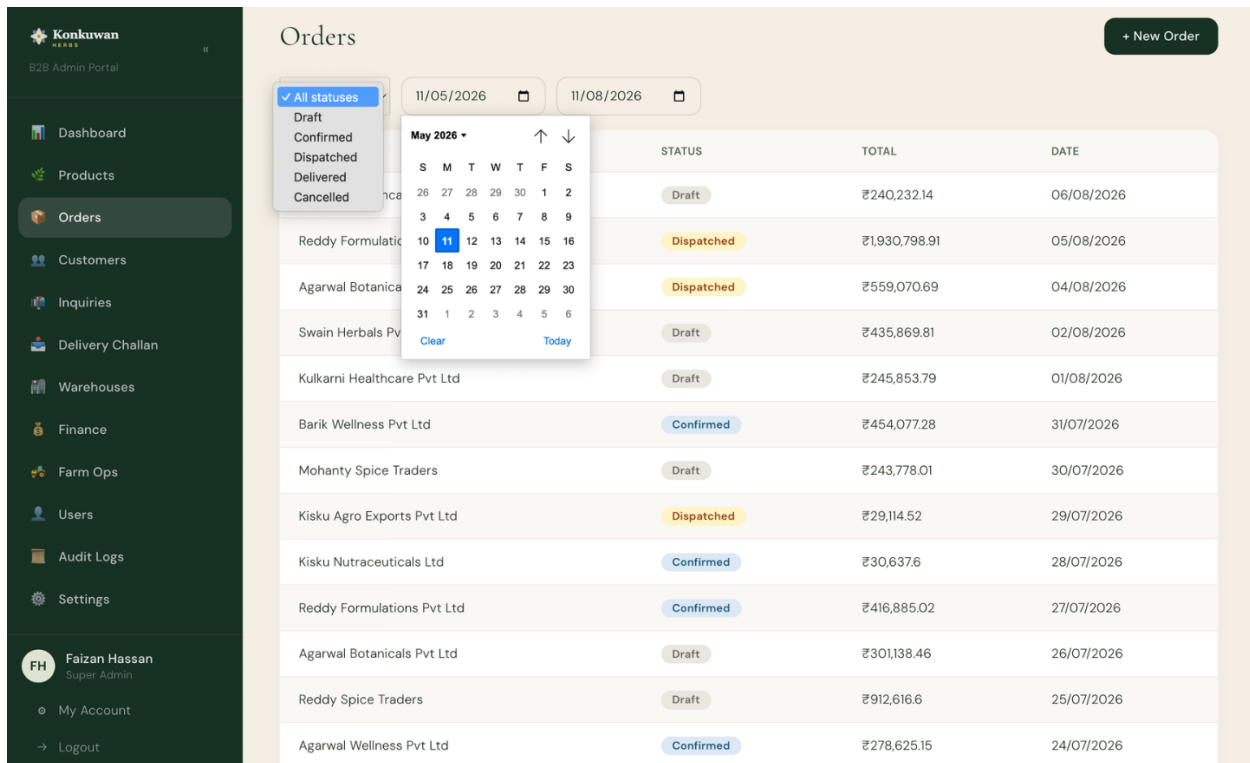


Figure 8.5.1: Order Management: Orders List with Status Filters

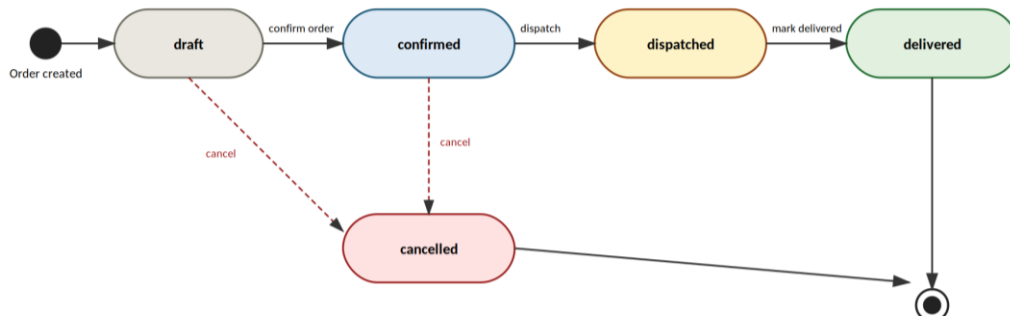


Figure 8.5.2: Order status state diagram, enforced server-side by the allowed-transitions map

The negotiated-pricing workflow is handled by a dedicated endpoint, `PUT /api/admin/orders/:id/items/:itemId/final-price`. Setting a final price on a line item updates that item, writes a row to `pricing_history` recording the negotiated amount against the relevant order, and recalculates the order's `total_amount` using the final price where one has been set, or the original quantity-times-unit-price line total otherwise. Figure 8.2 traces this flow step by step.

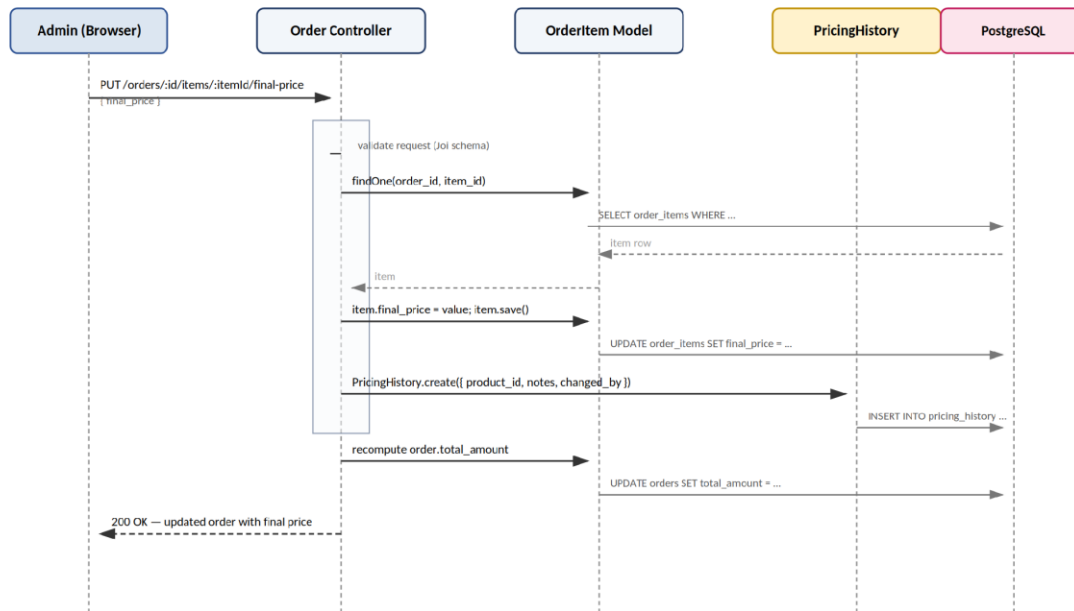


Figure 8.5.2: Sequence diagram for setting a negotiated final price on an order line item

Three additions since the original implementation:

- Off-catalogue line items.** A line may identify its product by `product_id` or by free-text `product_name`, so a one-off or seasonal crop can be sold without polluting the catalogue. Every consumer — invoice, quotation, order detail, customer profile, top-products analytics - resolves the display name through a single helper so they cannot disagree.
- Quotations.** GET `/api/admin/orders/:id/quotation` produces a quotation document with its own numbering sequence (`K/<FY>/K<n>`), and the resulting invoice references it.
- Shared billable-status definition.** The statuses that count as revenue (confirmed, dispatched, delivered) are defined once in `utils/orderStatus.js`. This resolved a defect in which the dashboard filtered on delivered alone while the Finance page used all three, so the two screens reported different revenue.

Order Details

#2ABE1AB8
Invoice: KON/2026-27/054

CUSTOMER: Agarwal Wellness Pvt Ltd
ORDER DATE: 24/07/2026

TOTAL AMOUNT: ₹278,625.15

NOTES: Buyer requested split dispatch across two lots.

LINE ITEMS

PRODUCT	QUANTITY	UNIT PRICE	FINAL PRICE	
Holy Basil (Tulsi)	7.243 MT	₹2898.28	₹20992.24	Set price
Dry Ginger	454.934 kg	₹199.48	₹90750.23	Set price
Moringa	415.648 kg	₹4015	Amount	Save X

Move to:

Dispatched Cancelled

Figure 8.4.2: Order detail with line items and the "Set price" control.

Once an order reaches a billable state, GET `/api/admin/orders/:id/invoice` assembles an invoice object (invoice number, customer details, line items with whichever price applies, and totals), which the

frontend renders into a downloadable PDF using jsPDF and jsPDF-AutoTable, including the company letterhead.

The screenshot displays the 'Orders' section of the Konkuwan Herbs Admin Platform. A modal window titled 'Order Details' is open, showing information for order #2ABE1AB8, dated 24/07/2026, for customer Agarwal Wellness Pvt Ltd. The total amount is ₹278,625.15. The modal includes buttons for 'Generate Quotation' and 'Generate Invoice PDF'. Below the modal, a table lists orders with columns for customer name, status, amount, and date.

The 'GST Invoice' PDF is also shown, detailing the invoice for the same order. It includes the 'Billed By' (Konkuwan Herbs) and 'Billed To' (Agarwal Wellness Pvt Ltd) information. The invoice lists three items: Holy Basil (Tulsi), Dry Ginger, and Moringa, with their respective quantities, unit prices, and final prices. The total invoice amount is ₹2,78,625.14. The PDF also includes bank details and terms and conditions.

Figure 8.5.2: Generated GST invoice PDF

The screenshot displays the 'Orders' section of the Konkuwan Herbs Admin Platform. A modal window titled 'Order Details' is open, showing information for order #2ABE1AB8, dated 24/07/2026, for customer Agarwal Wellness Pvt Ltd. The total amount is ₹278,625.15. The modal includes buttons for 'Generate Quotation' and 'Generate Invoice PDF'. Below the modal, a table lists orders with columns for customer name, status, amount, and date.

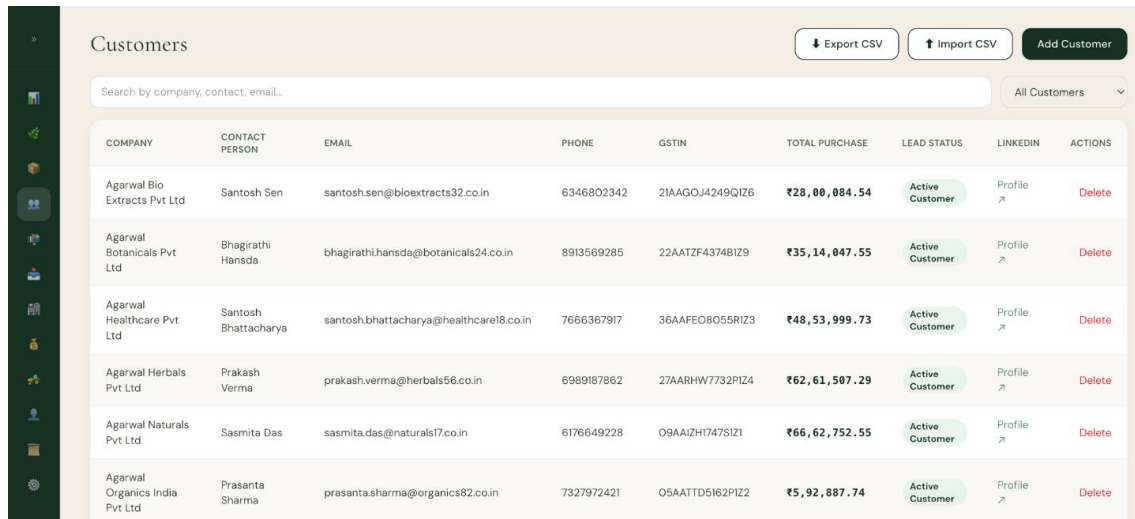
The 'Quotation' PDF is also shown, detailing the quotation for the same order. It includes the 'Quotation From' (Konkuwan Herbs) and 'Quotation For' (Agarwal Wellness Pvt Ltd) information. The quotation lists three items: Holy Basil (Tulsi), Dry Ginger, and Moringa, with their respective quantities, unit prices, and final prices. The total quotation amount is ₹2,78,625.14. The PDF also includes bank details and terms and conditions.

Figure 8.5.2: Generated quotation PDF

8.7 Customer Relationship Management

The customer module manages buyer companies, separate from the public-facing contact form. Each customer record carries a company name, contact person, GSTIN, address, and a lead_status field

distinguishing an active_customer from a potential_lead, along with an optional LinkedIn URL for quick reference during follow-up. This lead-status distinction was added specifically because Konkuwan's sales process involves a long pipeline of inquiries that may take months to convert, and the company needed a way to separate live customers from prospects still being worked.



COMPANY	CONTACT PERSON	EMAIL	PHONE	GSTIN	TOTAL PURCHASE	LEAD STATUS	LINKEDIN	ACTIONS
Agarwal Bio Extracts Pvt Ltd	Santosh Sen	santosh.sen@bioextracts32.co.in	6346802342	21AAGOJ4249QIZ6	₹28,00,084.54	Active Customer	Profile ↗	Delete
Agarwal Botanicals Pvt Ltd	Bhagirathi Hansda	bhagirathi.hansda@botanicals24.co.in	8913569285	22AATZF4374BIZ9	₹35,14,047.55	Active Customer	Profile ↗	Delete
Agarwal Healthcare Pvt Ltd	Santosh Bhattacharya	santosh.bhattacharya@healthcare18.co.in	7666367917	36AAFE08055RIZ3	₹48,53,999.73	Active Customer	Profile ↗	Delete
Agarwal Herbals Pvt Ltd	Prakash Verma	prakash.verma@herbals56.co.in	6989187862	27AARHW7732PIZ4	₹62,61,507.29	Active Customer	Profile ↗	Delete
Agarwal Naturals Pvt Ltd	Sasmita Das	sasmita.das@naturals17.co.in	6176649228	09AAI2H1747SIZ1	₹66,62,752.55	Active Customer	Profile ↗	Delete
Agarwal Organics India Pvt Ltd	Prasanta Sharma	prasanta.sharma@organics82.co.in	7327972421	05AATT5162PIZ2	₹5,92,887.74	Active Customer	Profile ↗	Delete

Figure 8.1.1: Customer list with lead-status badge and Total Purchase column.

Because the company already had an informal list of contacts before this system existed, the module includes a bulk CSV import path using PapaParse on the frontend. Imported rows are validated against the same Joi schema used for manual entry, and duplicates are rejected if an existing customer shares the same company name or email, with a per-row summary of how many records were imported, skipped, and why.

Additions since the original implementation: a **customer profile page** showing order history, products purchased, first/last order dates and lifetime spend, a **purchase-analytics bar chart** sortable by value, order count or name with paged navigation, and a **Total Purchase** column in the list.

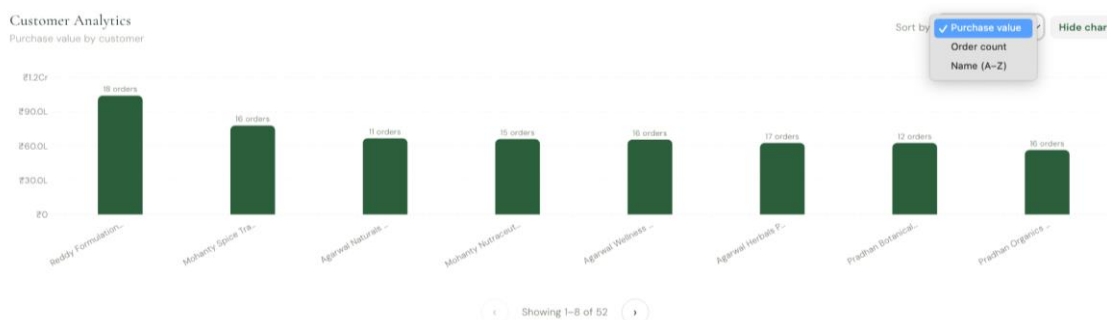


Figure 8.1.1: Purchase-analytics bar chart, sortable.

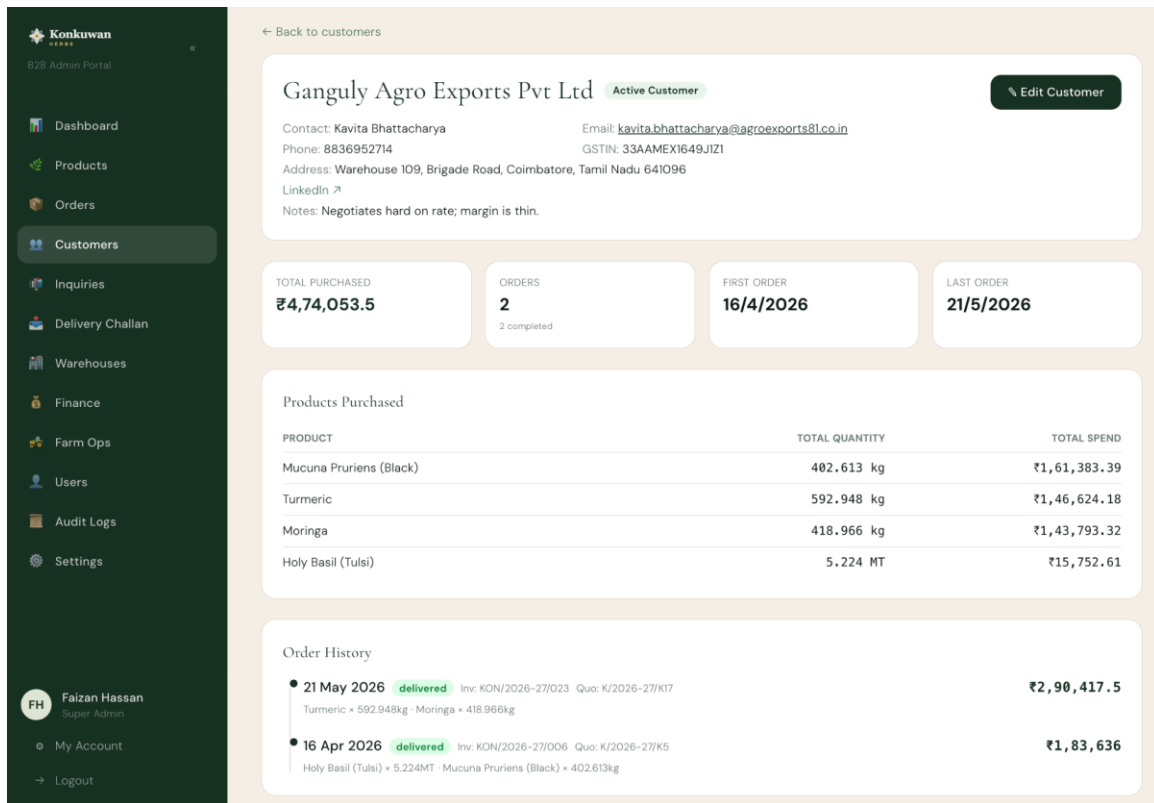


Figure 8.1.1: Customer profile with order history

8.8 Delivery Challans and Warehouses

The procurement module produces the document that had been missing entirely: a formal record of crops bought from a farmer.

Two types share one table.

Type	Purpose	Required fields
farmer_to_warehouse	Procurement	A farmer linked or free-text, and a destination warehouse
warehouse_transfer	Internal stock movement	Both warehouses, which must differ. No purchase is recorded

The “Other” farmer. Konkuwan occasionally buys from a grower who is not enrolled. Requiring enrollment first would either block the purchase or pollute the farmer registry with one-off entries. The challan therefore accepts a free-text farmer_name and farmer_address recorded on that document only.

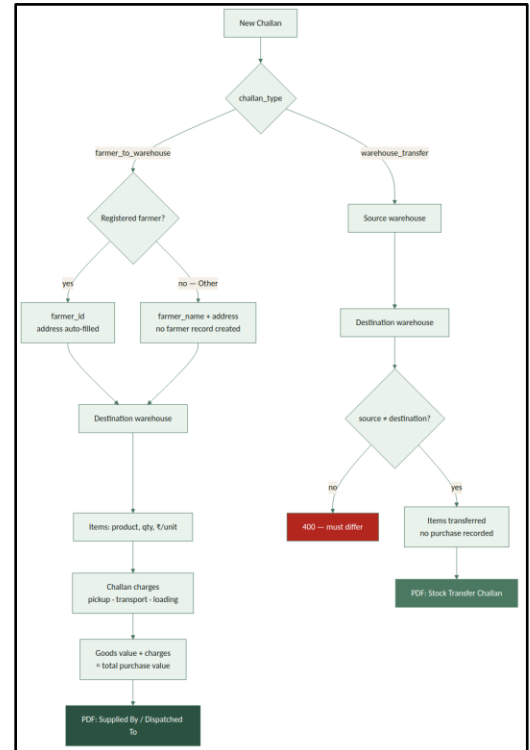


Figure 8.4.2: Challan type decision flow required fields and validation branches per type

Konkuwan
B2B Admin Portal

Dashboard
Products
Orders
Customers
Inquiries
Delivery Challan
Warehouses
Finance
Farm Ops
Users
Audit Logs
Settings

FH Faizan Hassan
Super Admin
My Account
Logout

Delivery Challan

Record purchases from farmers (crop pickup) before inventory is updated, including logistics charges.

Search by challan #

CHALLAN #

CHALLAN #	CHARGES	TOTAL	ACTIONS
CH/2026-27/019	₹2,500	₹8,54,128.26	Print Delete
CH/2026-27/018	₹3,200	₹8,65,033.87	Print Delete
CH/2026-27/017	₹800	₹9,16,591.06	Print Delete
CH/2026-27/016	₹800	₹76,185.3	Print Delete
CH/2026-27/015	₹1,200	₹1,72,406.83	Print Delete
CH/2026-27/014	₹6,000	₹59,254.12	Print Delete
CH/2026-27/013	₹600	₹1,57,437.72	Print Delete
CH/2026-27/012	₹1,800	₹91,782.02	Print Delete
CH/2026-27/011	₹6,000	₹34,386.68	Print Delete
CH/2026-27/010	₹600	₹9,701.71	Print Delete

New Challan

CHALLAN TYPE

Farmer → Warehouse
Goods procured from a farmer and received into a warehouse.

Warehouse → Warehouse
Stock moved between two warehouses. No purchase is recorded.

FARMER *
Other (not registered)

DESTINATION WAREHOUSE
Bhubaneswar Dispatch Hub

FARMER NAME *
Shivam tiwari

FARMER ADDRESS
Plot 42, Green Valley Road, Bhubaneswar

Recorded on this challan only — no farmer record is created.

DATE
11/08/2026

PRODUCTS PURCHASED *

Safed Musli	1540	228
Moringa	1100	138

+ Add product

CHALLAN CHARGES (₹)
5400

NOTES

Logistics: crop pickup, transport to warehouse, loading/unloading, other procurement costs.

Goods: ₹5,02,920 Total purchase value: ₹5,08,320

Cancel Create Challan

Figure 8.1.1: Farmer → warehouse challan with an unregistered (“Other”) supplier.

Konkuwan
B2B Admin Portal

Dashboard
Products
Orders
Customers
Inquiries
Delivery Challan
Warehouses
Finance
Farm Ops
Users
Audit Logs
Settings

FH Faizan Hassan
Super Admin
My Account
Logout

Delivery Challan

Record purchases from farmers (crop pickup) before inventory is updated, including logistics charges.

Search by challan #

CHALLAN #

CHALLAN #	CHARGES	TOTAL	ACTIONS
CH/2026-27/019	₹2,500	₹8,54,128.26	Print Delete
CH/2026-27/018	₹3,200	₹8,65,033.87	Print Delete
CH/2026-27/017	₹800	₹9,16,591.06	Print Delete
CH/2026-27/016	₹800	₹76,185.3	Print Delete
CH/2026-27/015	₹1,200	₹1,72,406.83	Print Delete
CH/2026-27/014	₹6,000	₹59,254.12	Print Delete
CH/2026-27/013	₹600	₹1,57,437.72	Print Delete
CH/2026-27/012	₹1,800	₹91,782.02	Print Delete
CH/2026-27/011	₹6,000	₹34,386.68	Print Delete
CH/2026-27/010	₹600	₹9,701.71	Print Delete

New Challan

CHALLAN TYPE

Farmer → Warehouse
Goods procured from a farmer and received into a warehouse.

Warehouse → Warehouse
Stock moved between two warehouses. No purchase is recorded.

SOURCE WAREHOUSE *
Raipur Transit Store

DESTINATION WAREHOUSE *
Bhubaneswar Dispatch Hub

TRANSFER DATE
11/08/2026

PRODUCTS TRANSFERRED *

Safed Musli	1540	228
Moringa	1100	138

+ Add product

CHALLAN CHARGES (₹)
5400

NOTES

Stock moved between two warehouses. No purchase is recorded.

Goods: ₹5,02,920 Total purchase value: ₹5,08,320

Cancel Create Challan

Figure 8.1.1: Warehouse → warehouse stock transfer.

Logistics charges - pickup, transport to warehouse, loading and unloading - are captured separately from goods value, so cost of goods and cost of movement are distinguishable.

Validation. A conditional Joi schema branches on `challan_type`. A subtle defect surfaced here: `Joi.valid('warehouse_transfer')` also matches `undefined`, so legacy payloads sending no type at all were routed down the transfer branch and rejected. Adding `.required()` inside the `.when()` condition fixed it, caught only because a test asserted the legacy payload explicitly.

Delivery Challan

Record purchases from farmers (crop pickup) before inventory is updated, including logistics charges.

Search by challan number or farmer...

CH/2026-27/019
5/8/2026

FARMER: Umesh Kisku
VILLAGE: Rampur

PRODUCT	QUANTITY	RATE	TOTAL
Moringa	1339.093 kg	₹213.36	₹2,85,708.88
Dry Ginger	953.666 kg	₹123.49	₹1,17,768.21
Safed Musli	461.783 kg	₹970.48	₹4,48,151.17

Goods value: ₹8,51,628.26
Challan charges (logistics): ₹2,500
Total purchase value: ₹8,54,128.26

Delivery Challan

Challan No # CH/2026-27/019
Challan Date 5 August 2026

Supplied By (Farmer)
Umesh Kisku
Rampur, Bishnupur GP, Dist. Nayagarh, Odisha 752140
Phone: 9151296945

Dispatched To
Bhubaneswar Dispatch Hub
Plot 44, Sector A, Manchewar Industrial Estate

#	Product / Crop	Quantity	Purchase Rate	Amount
1	Moringa	1339.093 kg	Rs 213.36	Rs 2,85,708.88
2	Dry Ginger	953.666 kg	Rs 123.49	Rs 1,17,768.21
3	Safed Musli	461.783 kg	Rs 970.48	Rs 4,48,151.17

Total (in words): EIGHT LAKH FIFTY FOUR THOUSAND ONE HUNDRED TWENTY EIGHT RUPEES ONLY
Goods Value Rs 8,51,628.26
Challan Charges Rs 2,500
Total Purchase Value Rs 8,54,128.26

Challan charges cover crop pickup, transportation to warehouse, loading/unloading and other procurement costs.

Farmer / Supplier Signature _____
Authorized Signatory _____

Figure 8.1.1: Printed delivery challan.

Konkuwan Herbs

B2B Admin Portal

Dashboard
Products
Orders
Customers
Inquiries
Delivery Challan
Warehouses
Finance
Farm Ops
Users
Audit Logs
Settings

Warehouses

WAREHOUSE NAME	CODE	LOCATION	CONTACT PERSON	STATUS	ACTIONS
Bhubaneswar D				Active	Deactivate Delete
Main Warehouse				Active	Deactivate Delete
Nimapada Colle				Inactive	Delete
Puri Processing				Active	Deactivate Delete
Raipur Transit S				Active	Deactivate Delete

Add Warehouse

WAREHOUSE NAME *
Main Warehouse Dehradun

CODE
WH-DRN

CONTACT PERSON
Shivam Verma

ADDRESS
Taria Nagal, Kulhan, Near Sahastradara Helipad, Dehradun (248013)

CITY
Dehradun

STATE
Uttarakhand

PIN CODE
248013

PHONE
9823935325

NOTES

Cancel Save

Figure 8.1.1: Warehouse registry.

8.9 Finance

FinanceOS is a lightweight bookkeeping layer for the farm side of the business, separate from the customer-order revenue tracked in the core schema. A single expenses table holds both expense and revenue rows, distinguished by a type column, with a fixed set of expense categories (labour, procurement, travel, seeds and inputs, processing, admin, rent, other) chosen to match how the company already categorised its spending informally.

Each expense entry can optionally carry a scanned receipt, uploaded directly to a Supabase Storage bucket from the browser, the resulting public URL is stored alongside the expense row and shown as a clickable receipt link in the transaction history. A separate cash-position panel lets the admin record the company's current cash on hand and compares it against twice the upcoming loan EMI to flag whether spending further is currently safe. Cash position now keeps a full **update history** (timestamp, old → new value, and the user who changed it).

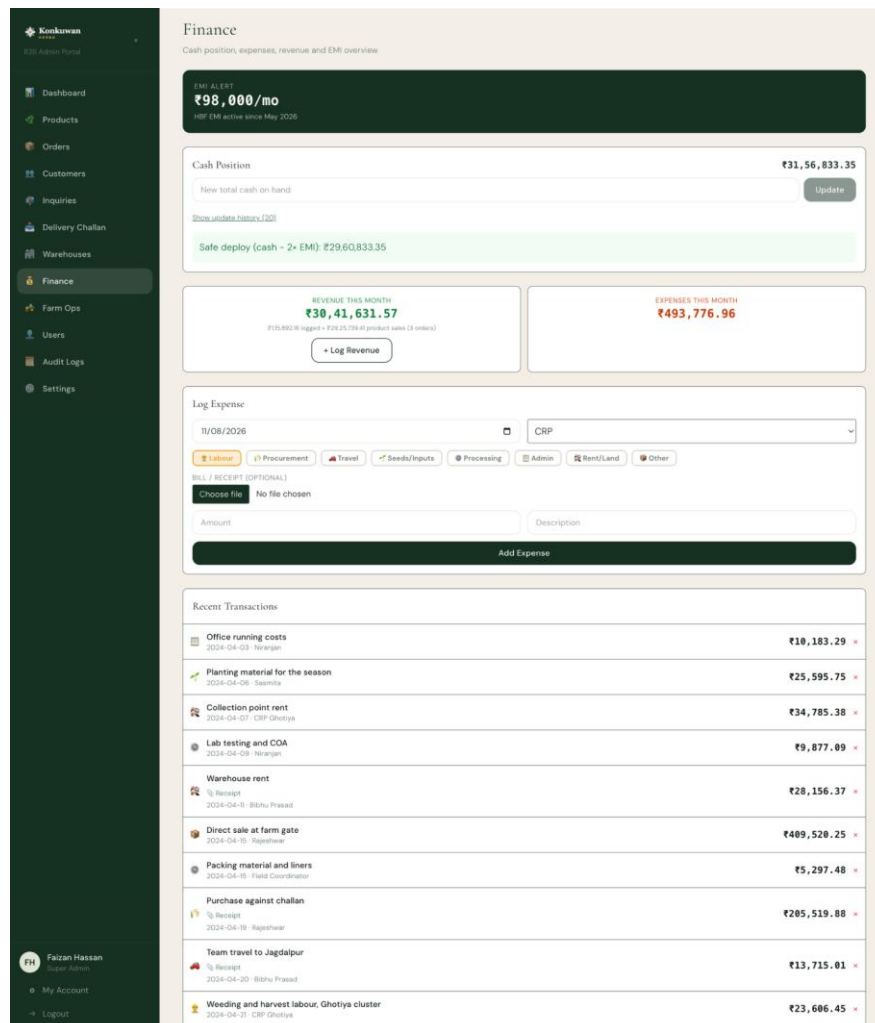


Figure 8.1.1: Warehouse registry.

8.10 Farm Operations — CropOS

Farm Operations was scoped into the project from the start, alongside the core admin panel, because Konkuwan’s business depends as much on what happens in the field as on what happens in sales. CropOS tracks four crops grown on the company’s pilot plots, Safed Musli, Moringa, Mucuna, and Ginger. Each with a fixed acreage and an expected harvest window defined in the frontend. Crops derive from products, so adding a product makes it available as a crop automatically.

For each crop, setting a `planting_date` starts a week counter calculated client-side from the current date. A weekly milestone map (defined per crop) tells the admin what task is due at a given week number — stand-count and gap-filling at week 4, first weeding at week 8, and so on. On top of this, the Generate POP button calls `POST /api/admin/farm/crops/:cropId/pop`, which builds a prompt containing the crop name and the number of weeks since planting and sends it to the configured AI provider, asking for a numbered list of concrete field tasks for that specific week. The result is stored as JSONB on the `crop_setups` row and displayed directly in the interface. Field staff can additionally log a weekly observation — plant health, pest pressure, water status, and growth stage — which is timestamped and listed in a running history.

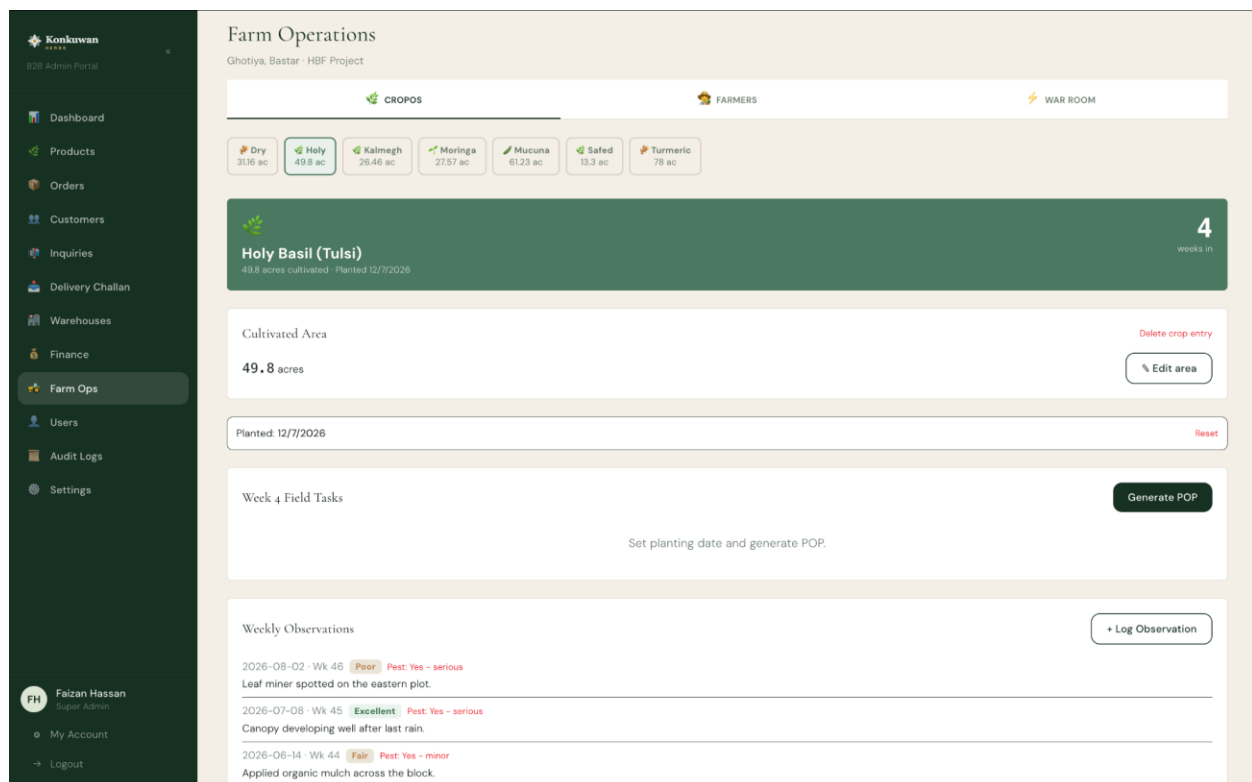


Figure 8.1.1: Warehouse registry.

8.11 Farm Operations — War Room

The War Room screen generates a weekly status brief intended for a Monday planning meeting. On request, the backend gathers the current state of all four farm-operations data sources, crop setups, expenses, farmers (with their visits), and the latest cash balance and sends a summarised version of that data to the configured AI provider with a system prompt instructing it to return a strict JSON structure: an overall GREEN, AMBER, or RED status, a one-line headline, a per-crop status list, a ranked list of top actions with owners, a risk radar, and an optional founder-decision flag.

The AI provider is **pluggable between OpenAI and Anthropic**, selected in Settings — the original implementation used only Anthropic’s API. Model output is treated as untrusted text: parsed and validated rather than assumed to be well-formed JSON, after early prompts returned JSON wrapped in markdown fences or preceded by a sentence of preamble.

The brief is parsed and stored in the `war_room_briefs` table as JSONB, tied to a week reference string supplied by the admin, and rendered in the interface as a colour-coded summary card followed by expandable sections for crop status, actions, and risks.

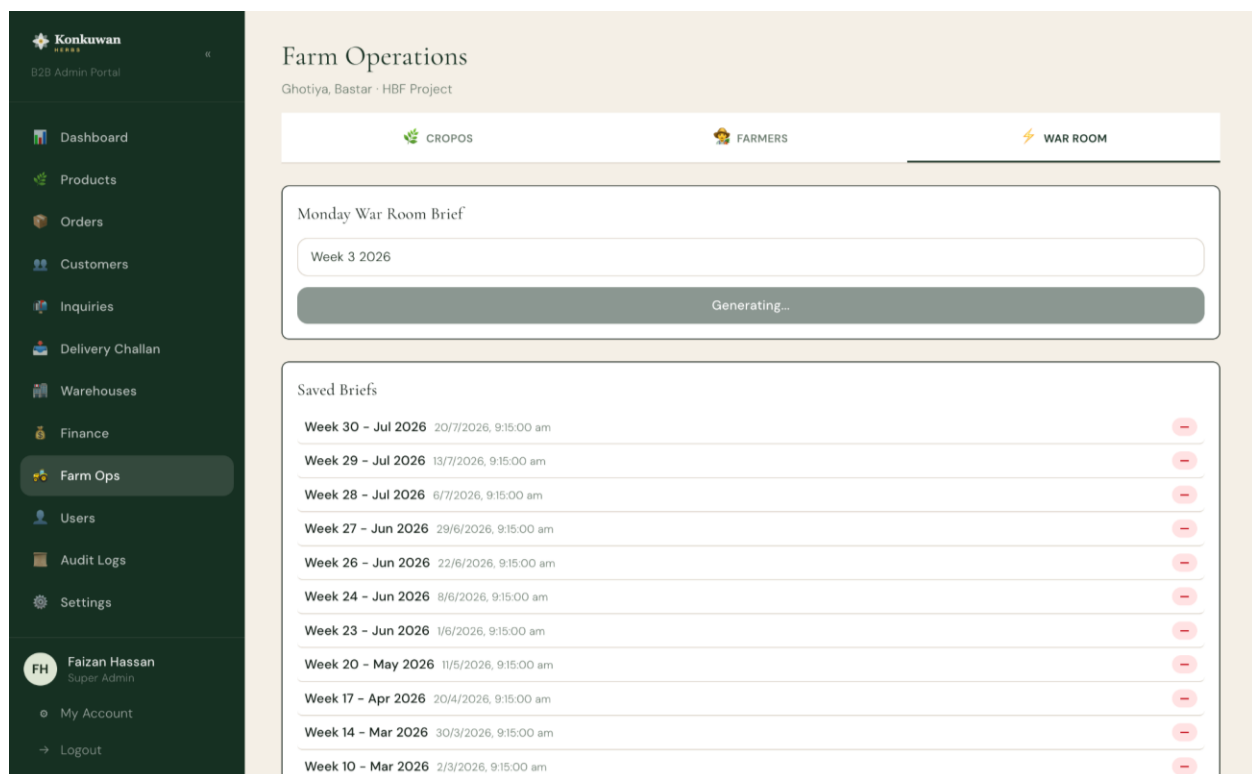


Figure 8.1.1: Warehouse registry.

8.12 Farm Operations — FarmerOS

FarmerOS tracks the farmers Konkuwan has enrolled into its cultivation programme against a fixed target per crop (1,000 for Moringa, 300 for Mucuna, 100 each for Ginger and Safed Musli). Coverage against these targets is shown as a set of progress bars on the module’s landing view, now with editable coverage

targets per crop. Enrolling a farmer records their name, village, block, assigned crop, cultivated area in decimals, and seeding date.

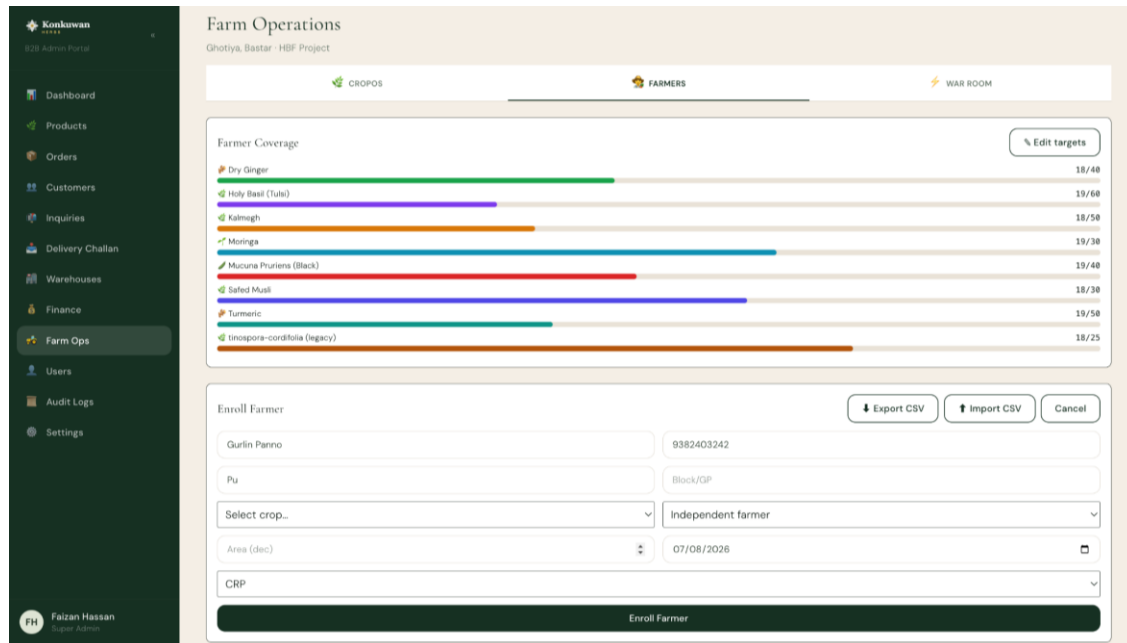


Figure 8.1.1: Farmer coverage bars per crop against enrolment targets, with crop filter buttons.

Each farmer can have field visits logged against them, recording a status (Excellent, Good, Fair, Poor — needs help, or Not planted) and free-text notes. The interface highlights any farmer whose most recent visit was more than two weeks ago with a different border colour, giving field coordinators a quick way to see who is due for a follow-up visit without cross-referencing dates manually. The module adds CSV import and export, two farmer types (independent, connected/contract), and a full farmer profile with visit history and activity timeline.

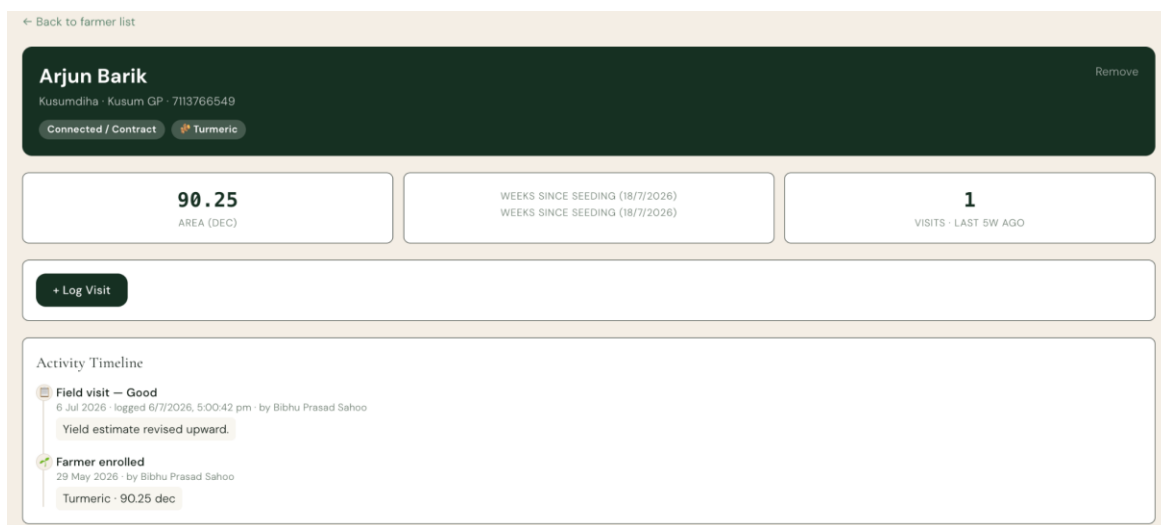


Figure 8.1.1: Farmer profile and activity timeline.

8.13 User Management and the Stakeholder Role

Restricted to super_admin. Beyond inviting and deactivating users, this module configures the stakeholder role.

The widget registry (utils/dashboardWidgets.js) is the single list of grantable metrics. The invite and edit screens fetch it from the server rather than hardcoding a copy, so the interface can never offer a widget the server does not honour.

Deliberate exclusions. The registry cannot grant cash position, EMI, individual expenses, audit logs, contact submissions, user management, or a farmer's personal details. These are not hidden by configuration — they are absent from the registry, so no administrator can grant them by accident.

A grant is a promise. Four designed widgets: customer analytics, farmer coverage, warehouse summary, inventory movement, reference data the dashboard payload does not produce. Rather than offer checkboxes that save a permission and render nothing, they were removed from the registry and documented as future work. The rule adopted: add the data first, then the key.

Figure 8.13.1: Stakeholder - Per-user dashboard widget grants

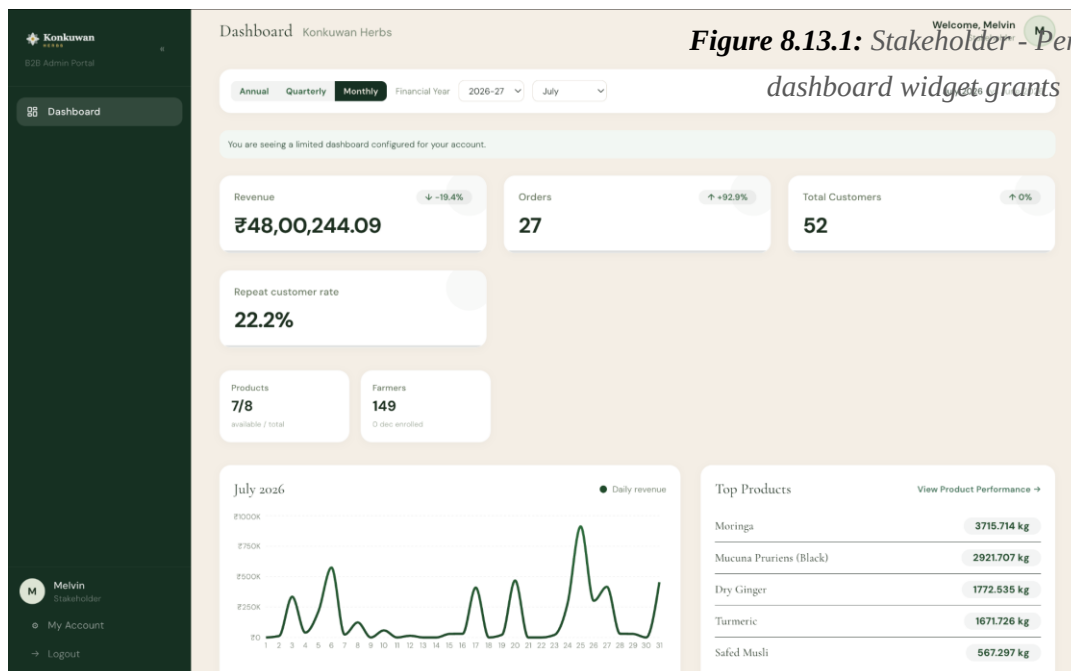


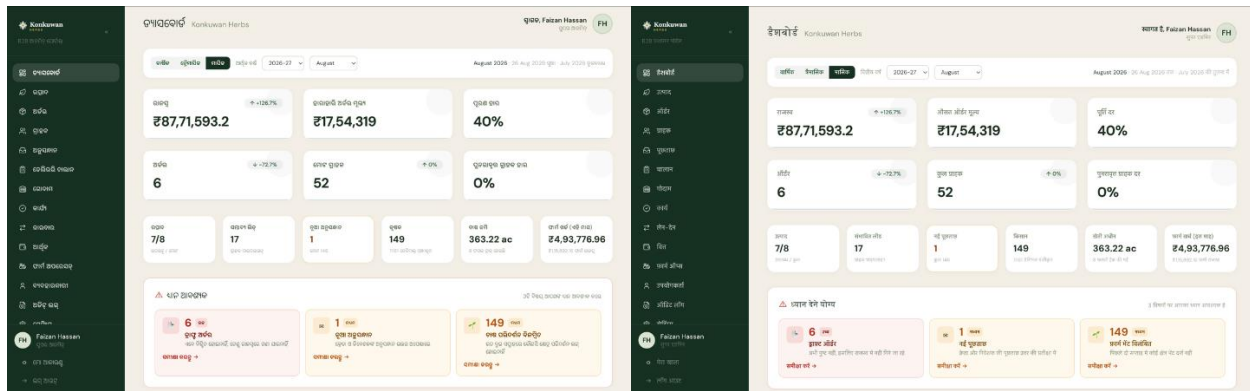
Figure 8.13.2: Restricted dashboard as seen by a stakeholder

8.14 Internationalisation — FinanceOS

The interface is available in English, Odia and Hindi, with 681 translated strings and full three-way parity.

Odia was the driving requirement: Konkuwan's farms and field staff are in Puri, Odisha. Hindi was added subsequently for broader reach.

Implementation. i18next with react-i18next, every user-facing string resolves through a `t()` call. The preference is stored on `profiles.language` and mirrored to `localStorage` so the first paint is already correct, before the session loads.



Statutory acronyms stay in Latin script. GSTIN, PAN, IFSC and IGST are legal identifiers and are not transliterated in any locale.

Figure 8.14.1: Admin interface in Odia.

Figure 8.14.2: Admin interface in Hindi.

PDF documents deliberately remain English.jsPDF performs no Indic text shaping, so Odia and Devanagari matras and conjuncts render reordered — visibly wrong on an invoice, which is a legal document. This is gated in one place (`PDF_LANGUAGES` in `lib/invoice.js`). Producing a correct English invoice was judged better than a mis-rendered Odia one.

8.15 Audit Logs and Settings

Every meaningful mutation in the system — creating a product, changing an order's status, setting a negotiated price, deactivating a user — calls a shared audit utility (`utils/audit.js`) that writes a row into `audit_logs` containing the acting user, the action, the entity type and id, and JSONB snapshots of the values before and after the change. This was added specifically in response to the lack of any accountability trail in the previous informal workflow, where it was never possible to say with certainty who had changed a price or order status.

The Audit Logs screen presents this trail as a filterable, paginated table, with filters for entity type and action type, colour-coded by action (green for creation, red for deletion or deactivation, blue for updates), now with **date-range filters** and a **CSV export** that exports every row matching the current filters rather than the page on screen, capped at 5,000 rows with a truncation notice. Exporting the audit trail is itself

written to the audit trail. The export neutralises **CSV formula injection** — a cell beginning =, +, - or @ is executed by Excel and Sheets when the file is opened.

TIME	USER	ACTION	ENTITY	DETAILS
27/08/2026, 00:46:21	Faizan Hassan	UPDATE	User #94f88c47	Name: Shivam · Role: product_manager · Active: Yes
27/08/2026, 00:36:21	Faizan Hassan	UPDATE	User #94f88c47	Name: Melvin · Role: stakeholder · Active: Yes
22/08/2026, 15:13:28	Faizan Hassan	UPDATE	Contact #87e8f23b	Status: read
22/08/2026, 15:11:00	Faizan Hassan	UPDATE	Settings	Bank ifsc: UTIB0000464 · Bank name: Axis Bank · Emi label: HBF EMI
22/08/2026, 15:09:47	Faizan Hassan	UPDATE	Order #2815de64	1 field changed: Status
22/08/2026, 15:06:26	Faizan Hassan	UPDATE	Product	Start: 0 · Reordered: 8
22/08/2026, 15:06:23	Faizan Hassan	UPDATE	Product	Start: 0 · Reordered: 8
22/08/2026, 15:06:09	Faizan Hassan	UPDATE	Product	Start: 0 · Reordered: 8
22/08/2026, 15:06:02	Faizan Hassan	UPDATE	Product	Start: 0 · Reordered: 8
22/08/2026, 15:00:52	Faizan Hassan	UPDATE	User #94f88c47	Name: Tiwari · Role: order_manager · Active: Yes
22/08/2026, 14:58:46	Faizan Hassan	DEACTIVATE	User #94f88c47	Active: No
22/08/2026, 14:57:36	Faizan Hassan	UPDATE	Product #49eafb5e	1 field changed: Price (min)
22/08/2026, 14:56:15	Faizan Hassan	UPDATE	Product	Start: 0 · Reordered: 8

Figure 8.13.2: Restricted dashboard as seen by a stakeholder

A separate, much smaller Settings screen exposes the key-value settings table as a simple form, now including **bank details** (used on invoices) and **invoicing defaults** (payment terms, tax percentage).

My Account

Profile

Name: Faizan Hassan

Email: thefaizanhassan@gmail.com

Mobile number: +918292163205

Role: Super Admin

Your name, role and mobile number are managed by an administrator.

Language

Choose the language for the admin panel. Your choice is saved to your account.

English ଓଡ଼ିଆ (Odia) हिन्दी (Hindi)

Change Password

CURRENT PASSWORD NEW PASSWORD CONFIRM NEW PASSWORD

Update Password At least 8 characters.

Figure 8.13.2: Restricted dashboard as seen by a stakeholder

9. Security Audit

This chapter is likely to be the strongest material in the report for a technical reader. It demonstrates the ability to find defects in one's own work.

9.1 Method

A structured pre-production review was carried out across seven areas: authentication and authorisation, the API surface, the database layer, file uploads, the frontend, the dependency tree, and production hardening. Findings were recorded before any fix was attempted, and categorised Critical / High / Medium / Low. Full detail is in AUDIT.md in the repository.

9.2 Critical findings

9.2.1 PostgREST filter injection

Supabase's `.or()` accepts a raw filter string that becomes a comma-separated condition list in the request URL. User input was interpolated directly into it at eight call sites:

```
q=q.or(`source_warehouse_id.eq.${warehouse_id},destination_warehouse_id.eq.${warehouse_id}`)
```

A value containing a comma injects additional conditions. Supplying

```
warehouse_id = "00000000-0000-0000-0000-000000000000,id.not.is.null"
```

produces a filter in which `id.not.is.null` matches **every row**, removing the warehouse scope entirely.

Impact. An authenticated user could widen any scoped or search query to return rows outside the intended filter. Because the server uses a service-role key that bypasses row-level security, the filter string was the only mechanism limiting the result set. Search boxes reached this code path directly.

Fix. `utils/pgrst.js` constructs filter clauses from structured components: search terms are escaped and quoted as `ilike` literals, identifier parameters are validated as UUIDs, and any value containing PostgREST metacharacters in an unquoted position is rejected with a 400.

Verification. The injected payload is now contained inside a quoted literal — it becomes an ordinary search term rather than syntax.

9.2.1 Database error disclosure

Sixty-six controller call sites returned the raw Supabase error message to the client, and the error handler passed it through unchanged for every status code. PostgreSQL and PostgREST error text names tables, columns and constraints. The public contact form is unauthenticated, so this was reachable without any credential.

Impact. Schema disclosure — the reconnaissance step that makes an injection attempt worth mounting.

Fix. Server errors (5xx) now return a fixed sentence together with a short correlation identifier; the real message is written to the log against the same identifier. Client errors (4xx), whose messages are written for users, still pass through unchanged.

9.3 High-severity findings

Table 9.1 — High-Severity Findings

	Finding	Impact	Fix
1	Every admin route was mounted with an empty <code>allowedRoles</code> array, which the guard treated as “allow everyone”	Any authenticated user — including a stakeholder — could render User Management or Settings by typing the URL. The API refused the data, so it was not a breach, but forms and structure were visible	One role→route table mirroring the server’s <code>authorize()</code> lists, with an automated check comparing the two
2	The sidebar offered every staff role every menu item	Most links returned 403 on arrival	Same shared table
3	URLs rendered into href from customer and expense records	Stored XSS. Supabase keeps the session token in <code>localStorage</code> , so script execution on the origin is account takeover	Scheme allow-list client-side: Joi scheme restriction server-side (a bare <code>.uri()</code> accepts javascript:)
4	The SPA carried no security headers at all	Helmet is Express middleware, and only <code>/api/*</code> passes through Express — the pages that execute JavaScript never touched it	CSP generated into <code>dist/_headers</code> at build time with the Supabase origin baked in. Express sets the same policy when serving the SPA in a container
5	Six list endpoints had no limit ceiling and no page floor	?limit=10000000 requests the whole table into a Worker with a 128 MB memory cap	Shared <code>parsePagination()</code> clamping page ≥ 1 and limit to 1–100
6	Reporting ranges ran into the future with a mismatched baseline	Every annual trend figure was wrong	Clamping — see §8.3
7	Dependency vulnerabilities	Five high-severity advisories in the client tree	axios, postcss, form-data, dompurify, brace-expansion upgraded

9.4 A finding that could not be fixed

react-router-dom is at **7.18.2, the latest published version**, and the advisory range covers 6.0.0 – 8.2.0. No patched release exists, `npm audit fix` proposes 7.11.0, which is a downgrade into the same range.

Exposure was therefore assessed rather than assumed. Four of the five advisories require server-side rendering or React Server Components — `deserializeErrors()` hydration, `RSCErrorHandler`, `RSC-mode` CSRF, and a route-matching denial of service that requires a server performing the matching. This application is a client-only SPA using `BrowserRouter`, with no `renderToString`, `StaticRouter`, `hydrateRoot` or `createStaticHandler` anywhere in the source. The fifth, an open redirect via a backslash in `<Link to>` or `useNavigate()`, requires a user-controlled destination, every navigation target in the application is a string

literal or comes from a hardcoded menu table, and the single interpolated path uses a UUID from the application's own database.

9.5 Hardening applied

Control		Implementation
Content Policy	Security	script-src 'self', frame-ancestors 'none', Supabase origin allowed for connect-src/img-src
Transport		HSTS; upgrade-insecure-requests; TLS terminated by Cloudflare
CORS		Not required — single origin by design
Request size		10 kB JSON and urlencoded body limit
File uploads		Magic-byte format verification, 5 MB cap, sanitised object keys
CSV output		Formula-injection neutralisation on all three exports
Error handling		5xx redaction with correlation ids
Rate limiting		Cloudflare WAF rule — an in-process limiter on Workers is per-isolate and would enforce nothing

10. Testing

Testing was carried out primarily during Phase 6, though validation of individual modules happened continuously as each one was completed, given the on-site setup allowed for quick feedback from Mr. Dhavala on whether a given screen matched how the team actually worked. The project's tooling includes Jest and Supertest on the backend and Vitest with React Testing Library on the frontend, used for targeted checks on the parts of the system where a regression would be most costly to miss — chiefly the order pricing logic and core display components.

10.1 Testing Approach

Testing was carried out continuously throughout development rather than as a single final phase, using a combination of manual end-to-end testing, controller-level testing, validation matrices, route-authorisation checks, state-transition tests, and component-level rendering checks.

1. **Manual end-to-end testing** was performed against the deployed staging environment for every major admin workflow, including creating products, placing orders, setting negotiated/final prices, generating invoices, and importing customer data from CSV files. Workflows were verified using representative company data before features were considered complete.
2. **Controller-level testing against a mocked data layer** was used to exercise the real, unmodified controllers against deterministic datasets. The Supabase client was replaced through `require.cache` injection, allowing backend logic—particularly dashboard calculations and period-based arithmetic—to be tested independently of the live database.
3. **Validation matrices** were used to exercise each Joi schema against valid, boundary, malformed, and legacy input shapes. This confirmed that missing required fields, malformed UUIDs, and other invalid inputs were rejected at the validation layer before reaching the database.
4. **Route authorisation matrices** verified that role-based middleware correctly rejected requests from accounts without the required permissions. The Express application was booted with a stubbed authenticator, allowing admin endpoints to be tested systematically across different roles.
5. **Order status transition testing** explicitly covered invalid state changes, such as attempting to move an order directly from `draft` to `delivered`, confirming that invalid transitions were rejected by the backend rather than being prevented only by the frontend.
6. **Negotiated pricing** was tested by setting a final price on an order and verifying that the order's `total_amount` was recalculated correctly and that a corresponding entry was created in `pricing_history`.
7. **CSV import testing** included deliberately malformed and duplicate records, such as rows with missing company names or duplicate email addresses. These tests confirmed that errors were reported on a per-row basis without causing the entire import operation to fail.

8. **Component-level rendering checks** using Vitest and React Testing Library covered presentational components such as ProductCard, verifying that product names, price ranges, and other displayed values rendered correctly when supplied with representative sample data.

10.2 Representative Test Cases

Test Case	Steps	Expected Result	Status
Filter injection	Search with <code>a,is_active.eq.false</code>	Treated as a literal search term; no extra condition in the query	Pass
Scope escape	<code>warehouse_id=<uuid>,id.not.is.null</code>	400 — value rejected as a non-UUID	Pass
Error redaction	Force a 5xx in production mode	Generic message + <code>error_id</code> ; schema not disclosed	Pass
Pagination clamp	<code>?limit=10000000, ?page=-5</code>	Clamped to 100 and to page 1	Pass
FY clamping	Annual view on 6 Aug 2026	Range 1 Apr → 6 Aug; baseline 1 Apr → 6 Aug 2025	Pass
Future period	Q4 FY2026-27 requested in August	Empty range; all figures zero	Pass
Legacy challan payload	Create a challan with no <code>challan_type</code>	Accepted, defaulted to <code>farmer_to_warehouse</code>	Pass
Same-warehouse transfer	Source = destination	400 with an explanatory message	Pass
Off-catalogue line	Line with <code>product_name</code> only	Accepted; appears correctly on invoice and analytics	Pass
Blank product line	<code>product_name: " "</code>	Rejected at the API, not at the database constraint	Pass
Stakeholder scope	Call 12 admin endpoints as a stakeholder	200 on the dashboard only; 403 on the other 11	Pass
Widget filtering	Grant one widget, inspect the payload	Only that widget's fields present; cash, expenses and audit withheld	Pass
Locale parity	Compare key sets across en/or/hi	681 / 681 / 681, no missing or extra keys	Pass
RBAC parity	Compare the client route table with server <code>authorize()</code> lists	12 of 12 match	Pass
Container routing	Run the image layout; request each path class	SPA served; <code>/api/nope</code> 404s as JSON; a missing asset 404s rather than returning HTML	Pass

Table 10.2.1:— High-Severity Findings

10.3 Defects found by testing

Three defects were found by the tests themselves rather than by use, which is the case worth reporting:

1. **The legacy challan payload.** `Joi.valid('warehouse_transfer')` also matches undefined, so payloads with no `challan_type` were routed down the transfer branch and rejected. Found only because a test asserted the legacy shape explicitly.
2. **Blank strings satisfying `.or()`.** `Joi's .or()` tests key presence, so `product_name: ''` satisfied “one of these is required” and produces a line with no product. It reached the database constraint and surfaced as a raw PostgreSQL error rather than a message.
3. **Dead widget grants.** A cross-check between the widget registry and the actual dashboard payload showed five of seventeen widgets referencing fields the payload never emits — grants that would have saved a permission and rendered nothing.

10.4 Limitations

The project has no committed automated test suite. The verification described above was performed through purpose-built scripts run during development. The test runner (Vitest) is configured and its setup file now exists, and the core logic — financial-year arithmetic, revenue-series derivation, filter construction — was deliberately extracted into pure modules specifically to make it testable. Writing the suite remains outstanding and is listed first in Chapter 13.

10.5 Bug Fixing and Refinement

Phase 6 surfaced a handful of issues typical of a project built solo and iteratively — inconsistent invalidation of TanStack Query caches after a mutation, a mismatch between the image URL paths returned by the API and the base URL the frontend expected in production, and a few mobile-layout issues in the admin sidebar that only became apparent once the interface was tested on an actual phone rather than a resized browser window. Each of these was logged informally, fixed, and re-tested manually before the application moved into Phase 7 for final refinement and deployment.

11. Results and Screenshots

This chapter summarises the outcome of the internship and consolidates the figures referenced throughout Chapter 8. Each screenshot placeholder in Chapter 8 is numbered Figure 10.x and corresponds to the module discussed immediately above it, the full list with suggested placement is repeated here for reference.

11.1 Delivered outcome

Metric	Outcome
Admin modules delivered	16
API endpoints	98 across 16 route files
Database tables in use	23
User roles	6 , including an external stakeholder role
Languages supported	3 (English, Odia, Hindi) — 681 strings each, full parity
Public pages rebuilt	7
Document types generated	3 (GST invoice, quotation, delivery challan)
AI-assisted features	2 (Package of Practices, War Room brief)
Server code	~4,150 lines across 55 files
Client code	~8,000 lines across 52 components and pages
Initial bundle size	1,763 kB → 481 kB after route-level code splitting
Security findings resolved	2 critical, 7 high , verified through 15 structured checks
Error handling	5xx redaction with correlation ids

11.2 Summary of Delivered Outcome

By the end of Phase 7, the platform was deployed and in active use by Konkuwan Herbs to manage its catalogue of ten core products and its order pipeline. The public website was running on the rebuilt React codebase while preserving its original visual identity, and the admin panel had replaced the spreadsheet-and-WhatsApp workflow described in Chapter 2 with a single system covering products, orders, customers, users, and the Farm Operations module.

Metric	Outcome
Products managed in catalogue	10 active products with images and price ranges

Metric	Outcome
Admin modules delivered	12 (Products, Categories, Orders, Customers, Users, Dashboard, Audit Logs, Settings, CropOS, FinanceOS, FarmerOS, War Room)
Public pages rebuilt	7 (Home, Products, Supply, Impact, Partners, About, Contact)
User roles implemented	5 (super_admin, product_manager, order_manager, farm_manager, viewer)
Database tables	19 across the core and farm-operations schema
AI-assisted features	2 (weekly POP generation, War Room weekly brief)
Deployment status	Deployed and in use by the company

10.3 Figure Index

All figures are placed inline within Chapter 8 at the point where the corresponding module is discussed: Figures 10.1–10.2 with the Public Website (8.1); Figure 10.3 with Authentication (8.2); Figures 10.4–10.5 with the Dashboard (8.3); Figures 10.6–10.7 with Product Management (8.4); Figures 10.8–10.10 with Order Management (8.5); Figures 10.11–10.12 with Customer Management (8.6); Figure 10.13 with User Management (8.7); Figures 10.14–10.15 with CropOS (8.8); Figures 10.16–10.17 with FinanceOS (8.9); Figures 10.18–10.19 with FarmerOS (8.10); Figure 10.20 with War Room (8.11); and Figure 10.21 with Audit Logs (8.12).

12. Learning Outcomes

Working alone on a project of this size, end to end, taught a different set of lessons than a typical coursework assignment, where the requirements are usually fixed in advance and the database schema is often handed out as part of the problem statement.

1. **Negotiated pricing as a first-class schema concept.** Separating `unit_price` (quoted) from `final_price` (agreed), with an append-only `pricing_history`, was the single design decision that made the business's central commercial question answerable. Learning to recognise which business questions *need* their own column, rather than being derivable later, was a lasting lesson.
2. **Role-based access control is a cross-cutting concern, not a feature.** Wiring `authorize(...roles)` correctly on every route, and keeping the client's menu table in sync with it, required treating access control as infrastructure to be checked automatically rather than a per-route decision to remember each time.
3. **An audit trail has to be a utility, not a convention.** Relying on every controller to remember to log a change produces gaps. Wrapping the write path in a shared `logAudit()` call made the trail complete rather than aspirational.
4. **AI output is untrusted input.** The Farm Operations AI features taught that a language model's response — however well-prompted — must be parsed and validated exactly like any other external input, never trusted to be well-formed.
5. **A deployment target can dictate architecture.** Removing Sequelize was not a refactor for its own sake; Workers cannot open a TCP connection to PostgreSQL. Learning to read a constraint of the runtime and let it drive a design decision — rather than fighting it — was the most transferable lesson of the project.
6. **Auditing one's own code requires a different posture from writing it.** The filter-injection defect had been in the codebase for weeks and was written by the same person who found it. Finding it required deliberately adopting an attacker's question — “what happens if I put a comma in this?” — rather than the author's question, “does this work?”
7. **A verification method can hide the bug it is meant to catch.** A configuration regression was introduced and then masked because the verification passed environment variables on the command line rather than loading a `.env` file the way production does. The lesson is to verify the way the system actually runs, not the way that is convenient.
8. **Saying “not fixed, and here is why” is part of the work.** The react-router advisory has no available fix. The professional response was to assess exposure precisely and record the reasoning, not to omit the finding or claim a fix that was not applied.
9. **A grant is a promise, not a checkbox.** Four stakeholder widgets referenced data the dashboard did not produce. Removing them from the registry rather than shipping a

permission that renders nothing reinforced that a feature is only real once the data behind it exists.

10. **Building for a business that depends on the system changes the standard.** Konkuwan used the platform for real orders and real farmer payments during development, which meant a defect was never purely academic — it had a person and a rupee amount attached to it.

13. Challenges Faced

Challenge	Resolution
Running Express on Cloudflare Workers	Workers gained node:http, making httpServerHandler viable. Required removing every dependency on the filesystem and on native modules, the ORM, disk-based file storage and file-based logging all had to go.
A dependency whose browser field lied	iconv-lite@0.4 maps ./lib/streams to false for browsers, but its index.js calls that module whenever process.versions.node exists, true on Workers, which is neither browser nor Node. Resolved with an npm overrides bump.
Joi shipping its browser build	Joi's browser field points at a minified bundle with the TLD list stripped, so .email() throws at runtime. Aliased to the Node entry point in wrangler.jsonc, keeping validation identical in development and production
A self-inflicted configuration regression	Removing the dead ORM layer also removed an accidental dotenv.config() that other modules depended on. Masked during verification, as described in Chapter 12.
A drill-down that silently did nothing	The chart always bucketed by month; in a monthly period that is one point. The interaction was not broken so much as meaningless, which made it harder to diagnose than an error would have been.
Indic script in generated PDFs	jsPDF performs no text shaping, so Odia and Devanagari conjunct render reordered. Chose correct English over incorrect Odia on legal documents, a product decision that had to be argued rather than an engineering one
Two workflows sharing one table	Delivery challans needed different required fields per type without splitting the table. Solved with a conditional Joi schema and a database CHECK constraint, where the constraint is the real guarantee.

Challenge	Resolution
Negotiated pricing without losing history	Overwriting a single price field would have destroyed the ability to answer “quoted vs achieved.” Solved with separate unit_price/final_price columns and an append-only pricing_history table.
Keeping three locales in parity	Adding Hindi after Odia risked silent drift between locale files. A parity check comparing key sets across all three locale files was added and is now run before any translation change is considered complete

14. Future Enhancements

Priority	Enhancement
Critical	Commit the missing DDL for delivery_challans, challan_items and contact_submissions. Until then a new environment cannot be provisioned from the repository.
High	Automated test suite. The runner is configured and the core logic is already extracted into pure modules for this purpose
High	Inventory. The table exists but is not surfaced. Computing stock balances would also unlock the warehouse-summary and inventory-movement widgets currently withheld from stakeholders
Medium	Harvest and yield tracking, closing the loop from crop to stock.
Medium	Farmer payouts and full supply-side cost of goods.
Medium	Caching for dashboard aggregations as order volume grows
Low	Email and WhatsApp notification digests.
Low	Offline-capable PWA field mode for visit and observation logging.
Low	Weather-aware crop guidance and yield forecasting
Low	Follow-up questions against War Room data rather than a fixed-format brief.

14. Conclusion

This internship took Konkuwan Herbs from a static website with no backend to a production platform covering the full commercial cycle, catalogue, customer relationships, orders with negotiated pricing, quotations and GST invoices, together with the procurement documents that make cost of goods knowable, and a farm-operations suite covering crops, farmers, field finance and AI-assisted weekly guidance. The system is in daily use, is available in three languages, and is deployed at the edge as a single Cloudflare Worker.

The most significant technical decision remains treating negotiated pricing as a first-class database concept — separate quoted and agreed prices with an append-only history — rather than overwriting a single price field. It is what makes the business’s central commercial question answerable at all.

Two decisions made after the original internship period deserve equal weight. Moving to Cloudflare Workers required removing the ORM entirely, which was a case of a runtime constraint dictating architecture rather than a preference being indulged. And the pre-production security audit found two critical defects in the author’s own code — a filter-injection vector and database error disclosure — which is the clearest demonstration in the project that reviewing work adversarially is a distinct skill from producing it.

The report also records what is not finished: three tables have no committed schema, there is no automated test suite, and the inventory module remains unsurfaced. Stating these plainly is more useful to whoever maintains the system next than a conclusion claiming completeness would be.

Beyond the platform itself, the internship was a practical exercise in building software for a real business under real constraints — a fixed timeline, a single developer, a non-technical stakeholder, and a production database holding data the company depends on.

References

1. React documentation. <https://react.dev/>
2. Vite documentation. <https://vite.dev/>
3. Express.js documentation. <https://expressjs.com/>
4. Supabase documentation. <https://supabase.com/docs>
5. PostgreSQL documentation. <https://www.postgresql.org/docs/>
6. TanStack Query documentation. <https://tanstack.com/query/>
7. Tailwind CSS documentation. <https://tailwindcss.com/docs>
8. Recharts documentation. <https://recharts.org/>
9. Joi validation documentation. <https://joi.dev/>
10. Konkuwan Herbs Pvt. Ltd. company website. <https://konkuwanherbs.com>
11. Cloudflare Workers documentation — Node.js compatibility. <https://developers.cloudflare.com/workers/runtime-apis/nodejs/>
12. Cloudflare Workers Static Assets. <https://developers.cloudflare.com/workers/static-assets/>
13. PostgREST API reference — operators and filters. https://postgrest.org/en/stable/references/api/tables_views.html
14. OWASP Top Ten (2021). <https://owasp.org/Top10/>
15. OWASP CSV Injection. https://owasp.org/www-community/attacks/CSV_Injection
16. Content Security Policy Level 3, W3C. <https://www.w3.org/TR/CSP3/>
17. Docker best practices for Node.js. https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
18. i18next documentation. <https://www.i18next.com/>
19. OpenAI API reference. <https://platform.openai.com/docs/api-reference>
20. Anthropic API documentation. <https://docs.claude.com/>